



湖南大学
HUNAN UNIVERSITY

第十一章 最小生成树

—— 湖南大学计算机学院 ——

目录

第一节 最小生成树定义

第二节 Kruskal算法 (克鲁斯卡尔)

第三节 不相交集合应用

第四节 Prim算法 (普里姆)

第五节 斐波拉契堆

最小生成树定义

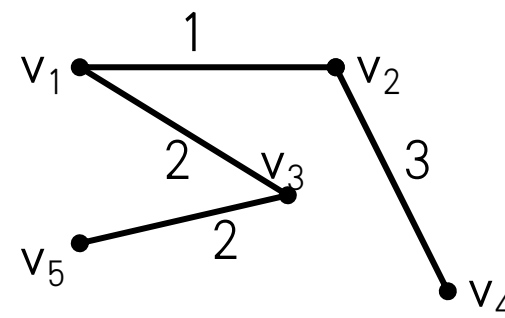
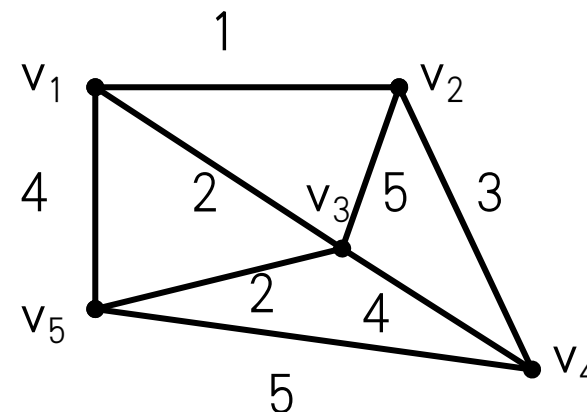


最小连接问题:

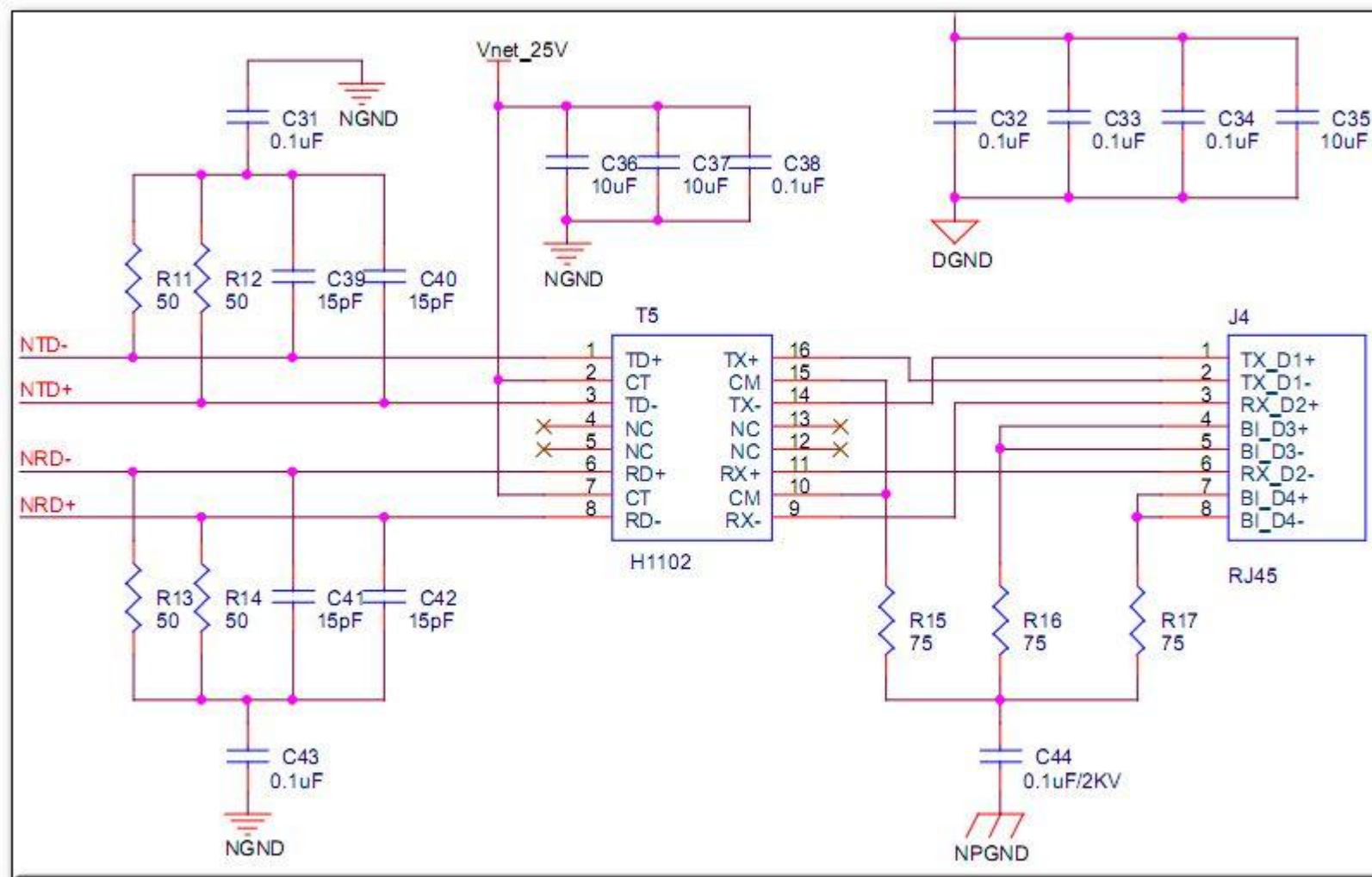
交通网络中，常常关注能把所有站点连接起来的生成树，使得该生成树各边权值之和为最小。

例如：

假设要在某地建造5个工厂，拟修筑道路连接这5处。经勘探，其道路可按下图的无向边铺设。现在每条边的长度已经测出并标记在图的对应边上，如果我们要求铺设的道路总长度最短，这样既能节省费用，又能缩短工期，如何铺设？



最小生成树定义

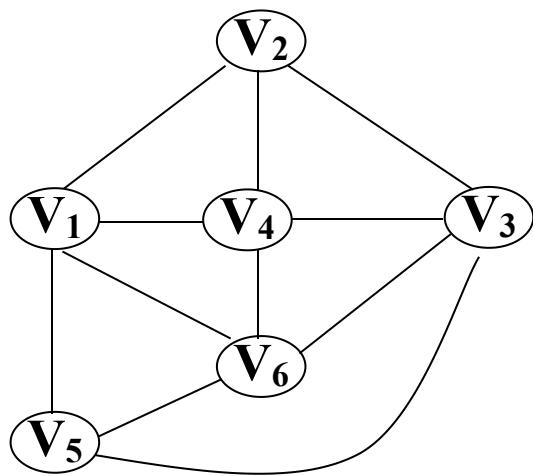


最小生成树定义

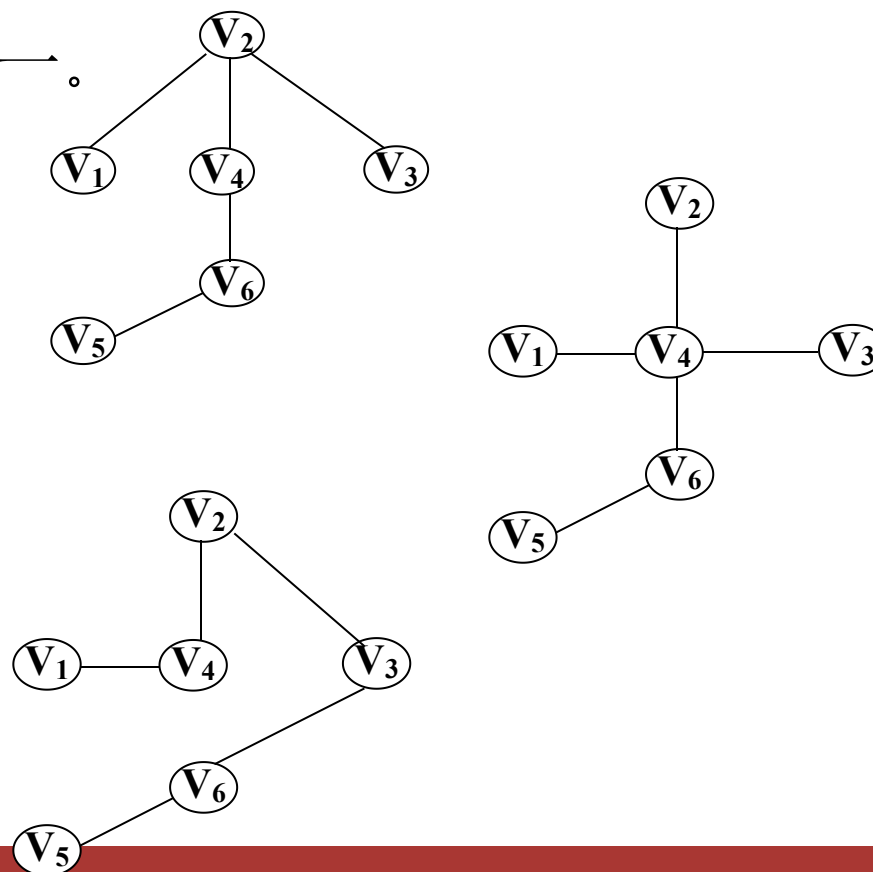


生成树定义:

一个连通图的生成树是一个极小连通子图，它含有图中全部顶点，但只有足以构成一棵树的 $n-1$ 条边。生成树不唯一。



生成树



最小生成树定义

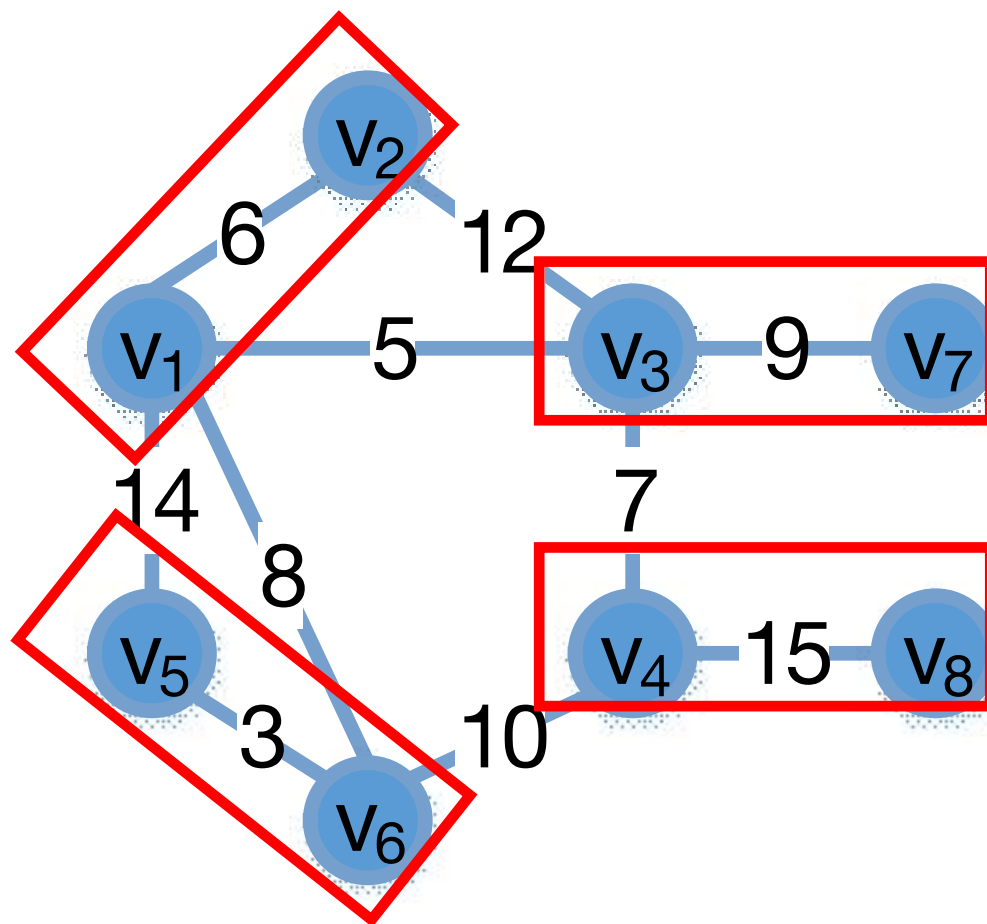


最小生成树定义 (Minimum Spanning Tree, MST)

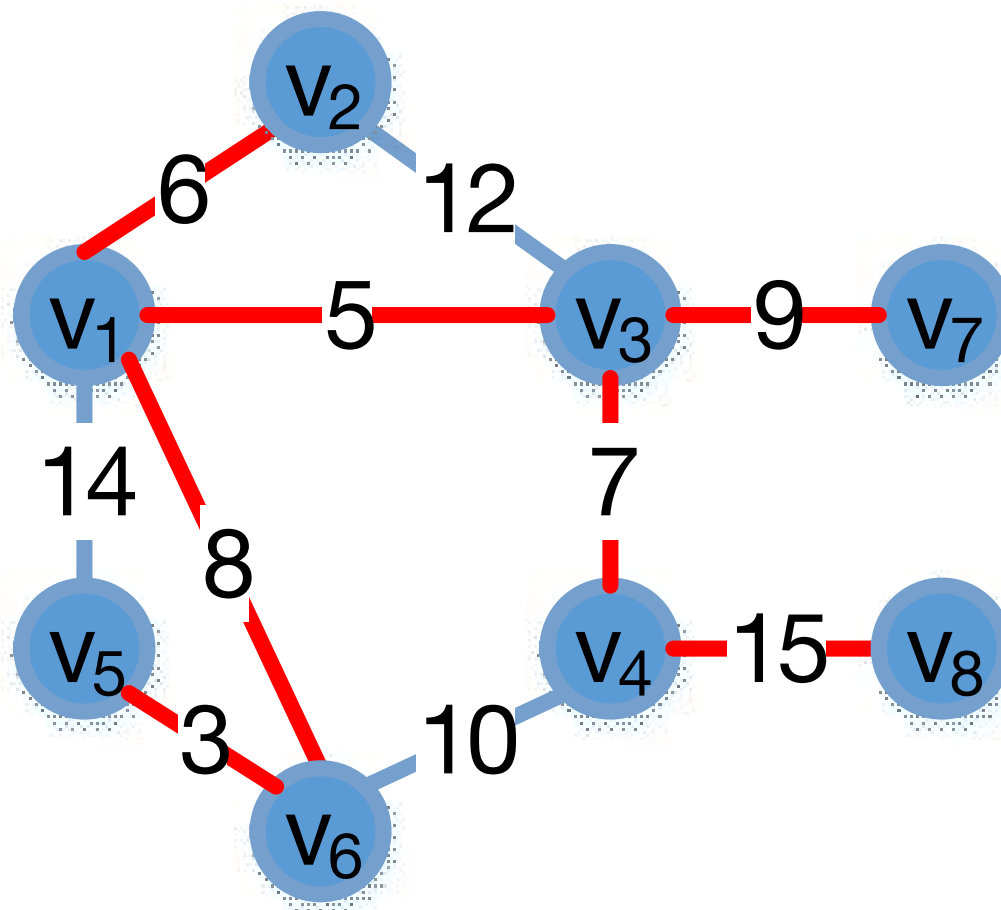
给定一个无向图 $G=(V, E, W)$ ，它的最小生成树 T 是它的极小连通子图，且包含 G 中的所有 $|V|$ 个结点，并且有保持整个树的权重之和最小，即

$$W(T) = \min_{T \subseteq G} \sum_{e \in T} w(e)$$

最小生成树定义



最小生成树定义



最小生成树定义



□ 两种常用的构造最小生成树的方法(贪心):

- Kruskal(克鲁斯卡尔)算法
- Prim(普里姆)算法



目录

第一节 最小生成树定义

第二节 **Kruskal**算法 (克鲁斯卡尔)

第三节 不相交集合应用

第四节 Prim算法 (普里姆)

第五节 斐波拉契堆

最小生成树定义



最优子结构性质

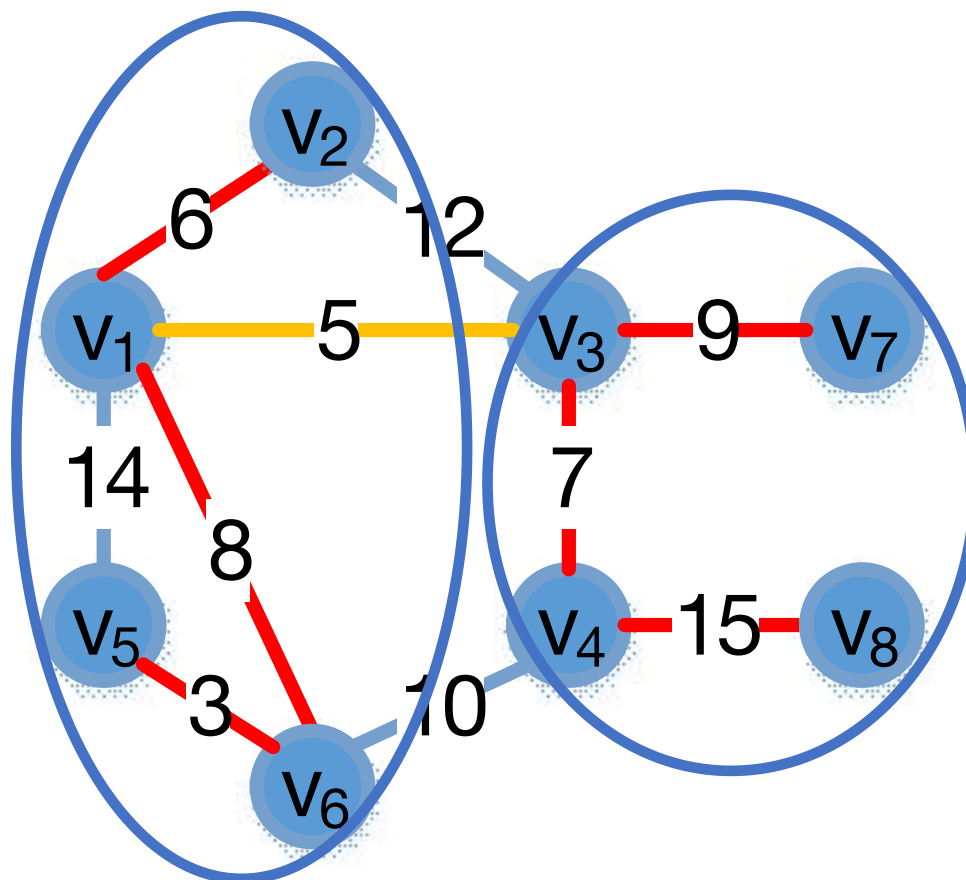
两个最小生成树通过其间的连接边合并后仍构成最小生成树： 给定一个无向图 $G=(V, E, W)$ 以及它的最小生成树 T ，如果从 T 中去掉一条边 (u, v) 进而将 T 划分成两个子树 T_1 和 T_2 ，那么 T_1 和 T_2 分别是图 $G_1=(V_1, E_1, W)$ 和 $G_2=(V_2, E_2, W)$ 的最小生成树，其中 V_i ($i=1$ 或 2) 是 T_i 中的点且 G_i 是 V_i 的导出子图。

证明： 根据 $W(T)=W(T_1)+W(T_2)+W(u, v)$ 进行反证

最小生成树定义



最优子结构示例

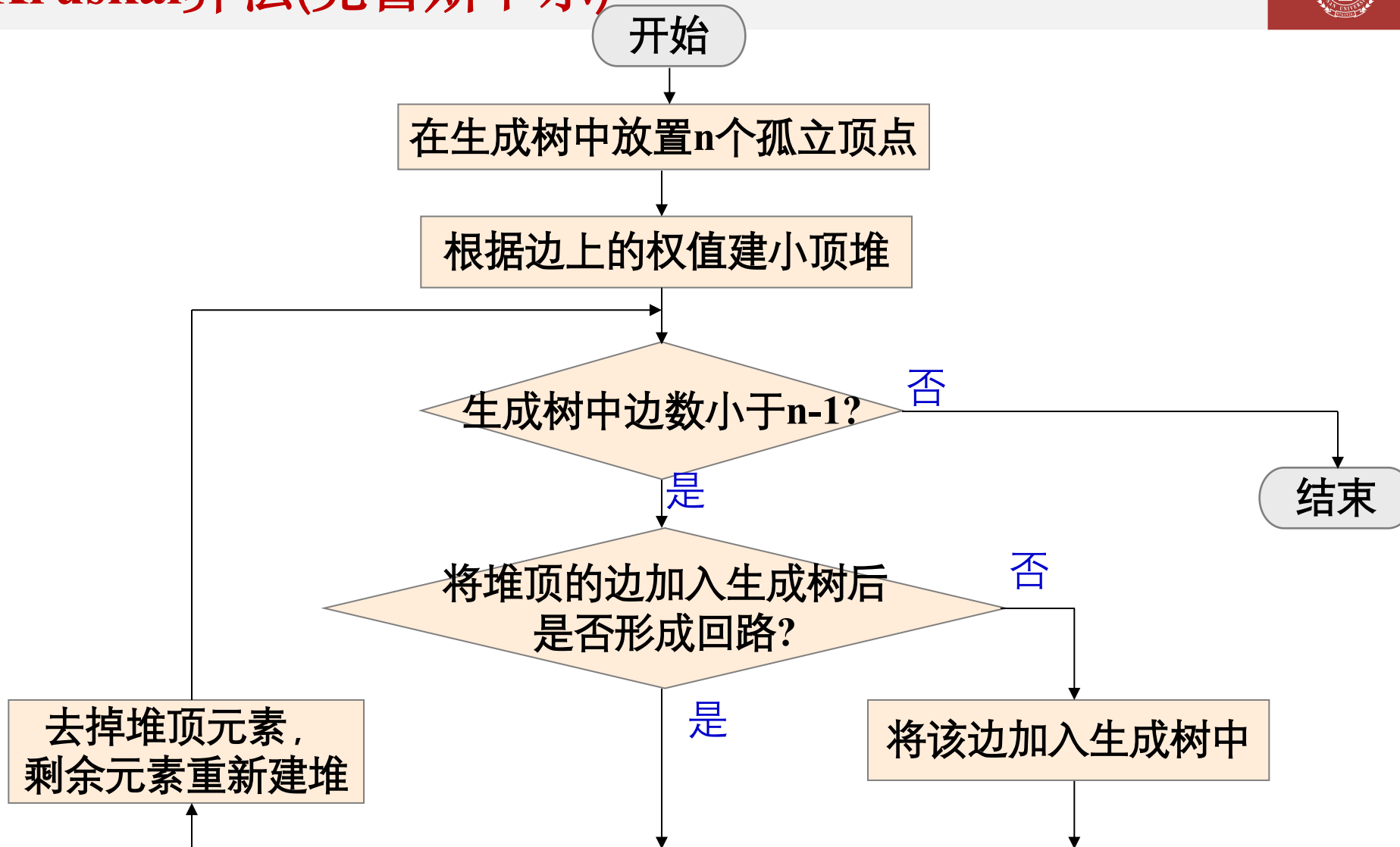


Kruskal算法(克鲁斯卡尔)



- 对于图 $G=(V, E, N)$, 则令最小生成树的初始状态为只有 n 个顶点而无边的非连通图 $T=(V, \{\})$, 图中每个顶点自成一个连通分量。
- 既然两个最小生成树通过其间的连接边合并后仍构成最小生成树, 那么在 E 中选择代价最小的边, 若该边依附的顶点落在 T 中不同的连通分量上, 则将此边加入到 T 中, 否则舍去此边而选择下一条代价最小的边。依次类推, 直至 T 中所有顶点都在同一连通分量上为止。

Kruskal算法(克鲁斯卡尔)



Kruskal算法(克鲁斯卡尔)



- 从上述过程可知，实现克鲁斯卡尔(Kruskal)算法时，要解决以下两个问题：
 - 如何选择代价最小的边(堆排序，或简单选择排序);
 - 如何判定边所关联的两个顶点是否在同一个连通分量中 (集合)



不相交集合数据结构的基本定义

1

定义与特性

不相交集合数据结构是一种树形数据结构,用于管理集合的合并与查询问题。

2

集合表示

每个集合通过一棵树表示,树的根节点作为该集合的代表。

3

集合元素

集合中的每个元素对应一个节点,节点间通过父子关系连接。



不相交集数据结构的内部结构

1

树形结构

集合以树的形式表示，通过父指针链接。

2

根节点

树的根节点代表整个集合，是操作的关键。

3

节点表示

每个节点代表一个元素，形成层次关系。

4

父指针

指向节点的父节点，根节点指向自己。

5

森林结构

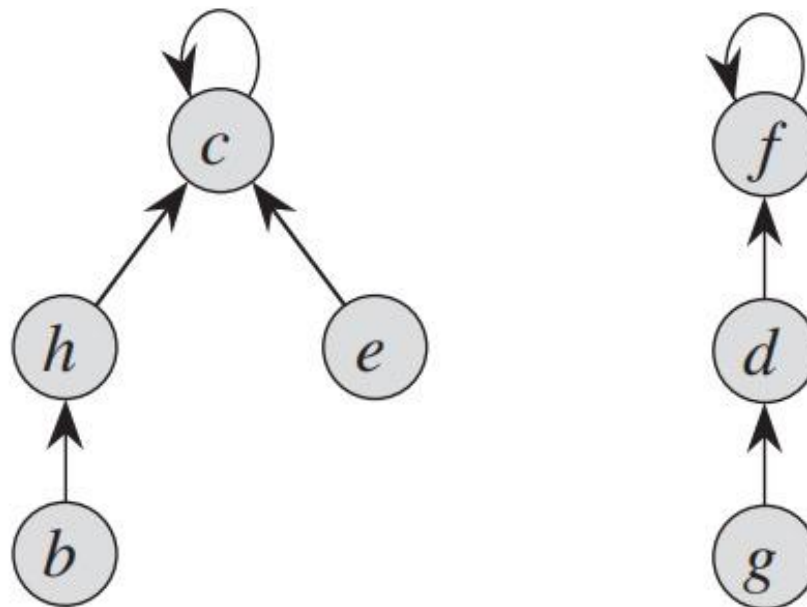
多个树构成森林，每个树为独立集合。

Kruskal算法(克鲁斯卡尔)



不相交集合数据结构的内部结构示例

两棵树分别代表两个集合。左侧的树代表集合 $\{b, c, e, h\}$ ，其中 c 是代表元素作为根节点；右侧的树代表集合 $\{d, f, g\}$ ，其中 f 是代表元素作为根节点。





不相交集合数据结构的基本操作

1

MakeSet

创建一个仅包含单一元素的新集合，每个元素独立成集。

2

Find

查找元素所在集合的代表，确定其根节点。

3

Union

合并两个不同集合为一个，更新集合关系。

Kruskal算法(克鲁斯卡尔)



不相交集合数据结构—基本算法

- **function** MakeSet(x)

 x.parent = x

- **function** Find(x)

 if x.parent == x

 return x

 else

 return Find(x.parent)

- **function** Union(x, y)

 xRoot = Find(x)

 yRoot = Find(y)

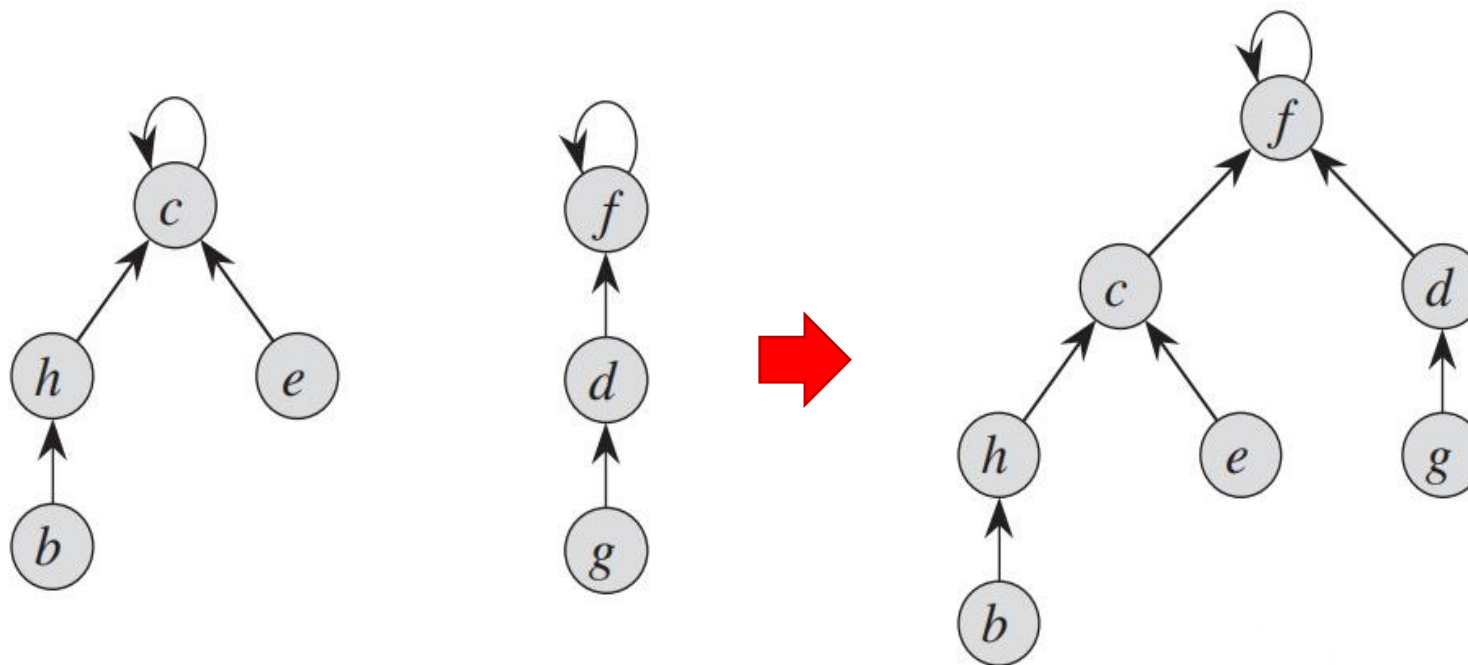
 xRoot.parent = yRoot

Kruskal算法(克鲁斯卡尔)



不相交集合数据结构的Union示例

合并Union(e,g)

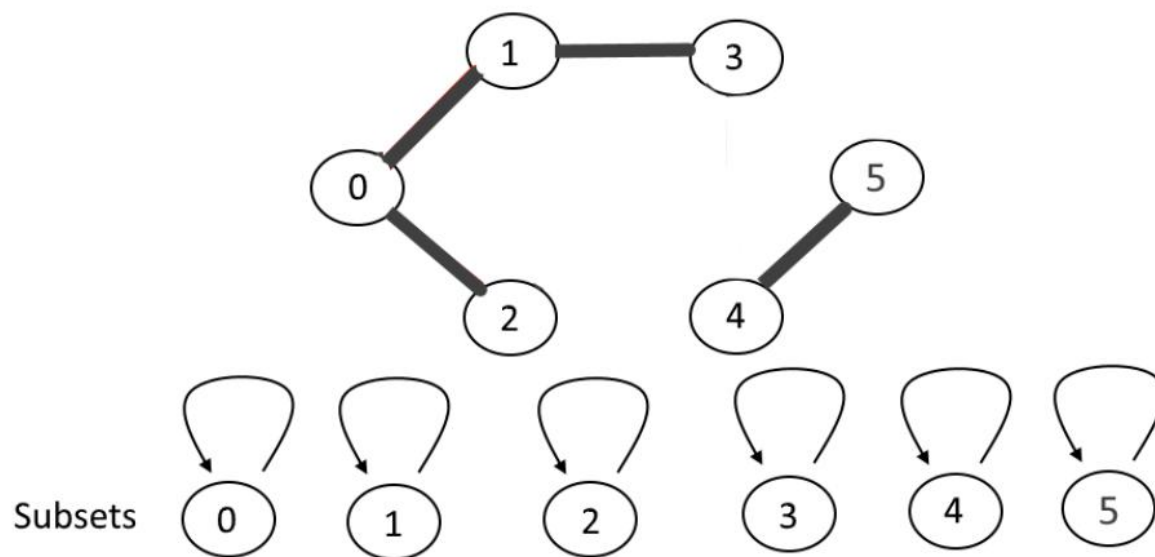


Kruskal算法(克鲁斯卡尔)



不相交集合数据结构-示例

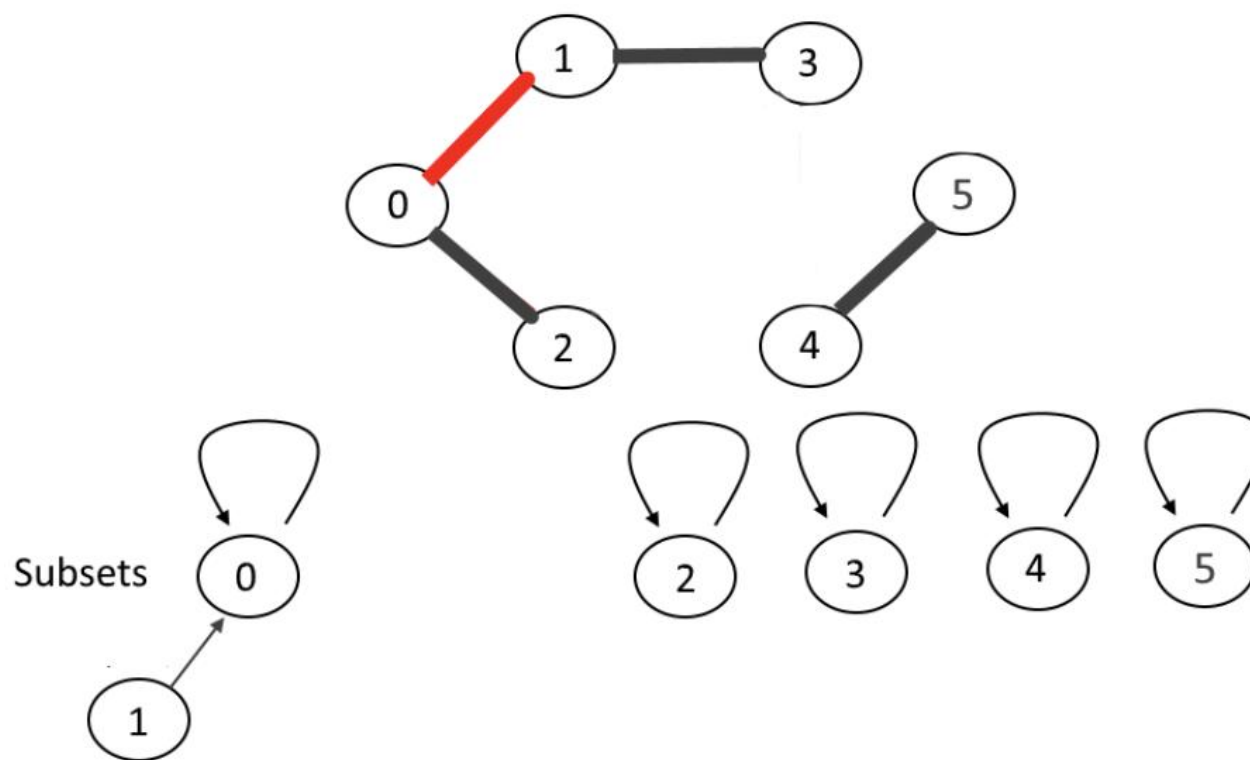
初始的时候每个点的parent都是自己



Kruskal算法(克鲁斯卡尔)



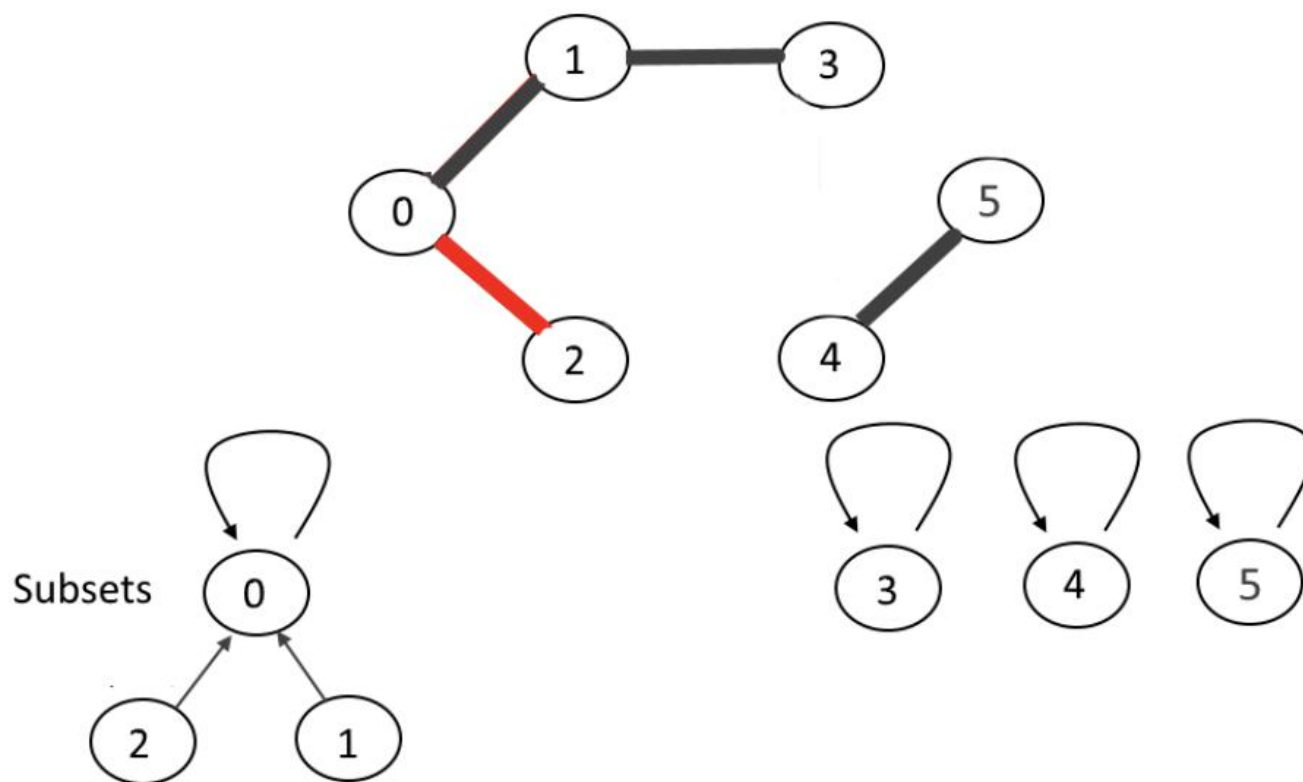
不相交集合数据结构-示例



Kruskal算法(克鲁斯卡尔)



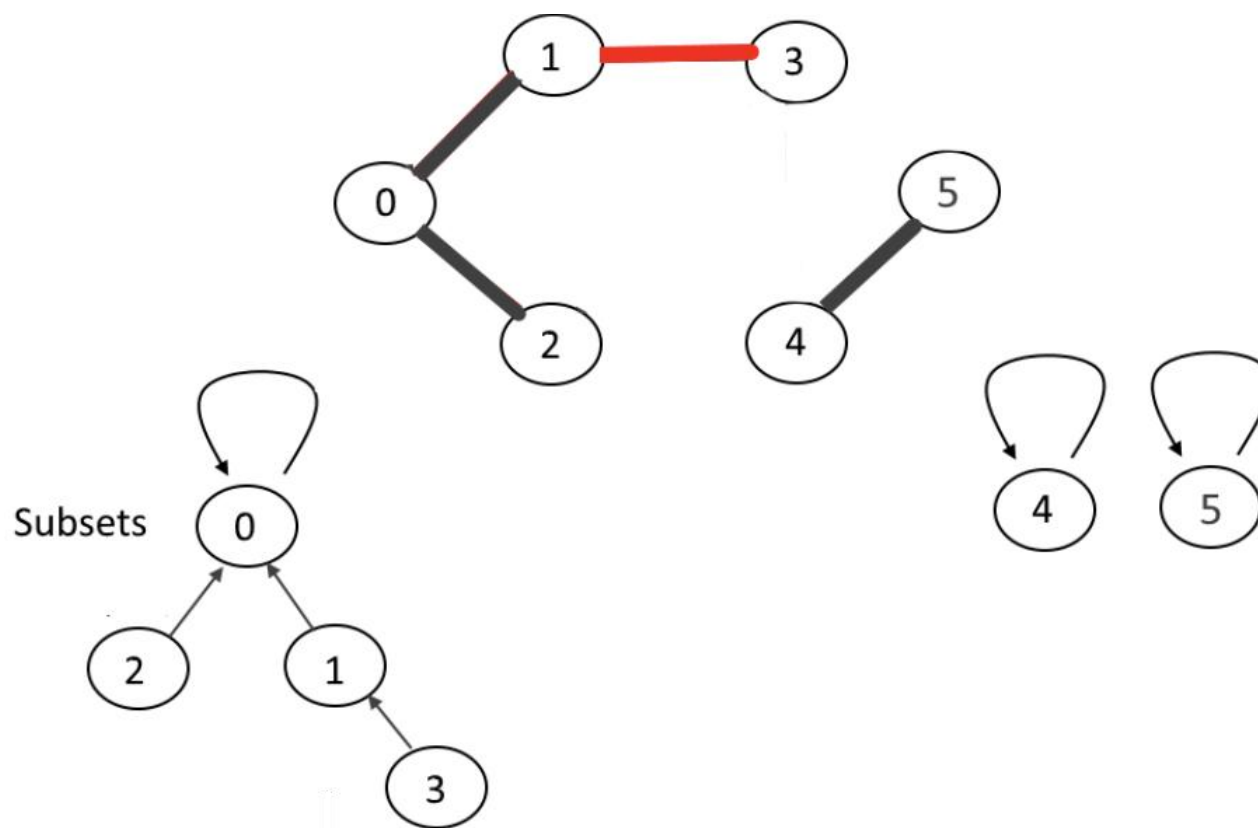
不相交集合数据结构-示例



Kruskal算法(克鲁斯卡尔)



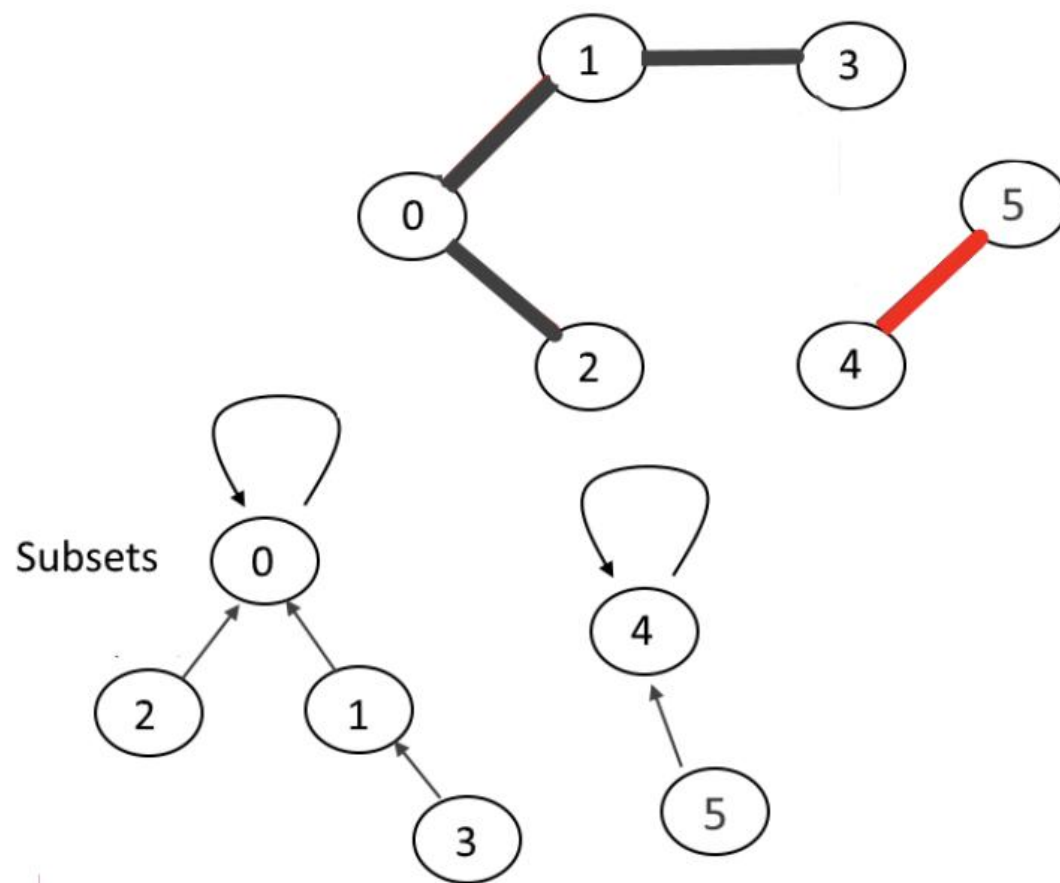
不相交集合数据结构-示例



Kruskal算法(克鲁斯卡尔)



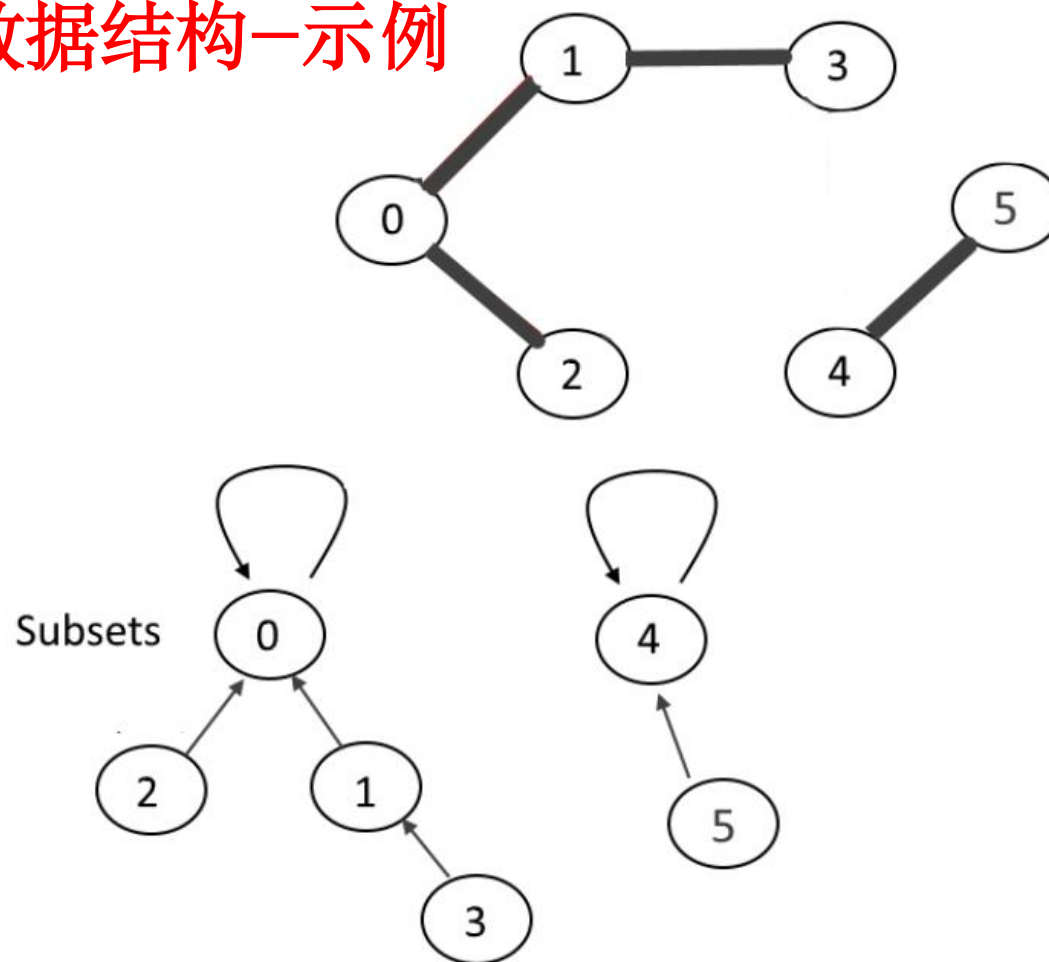
不相交集合数据结构-示例



Kruskal算法(克鲁斯卡尔)



不相交集合数据结构-示例

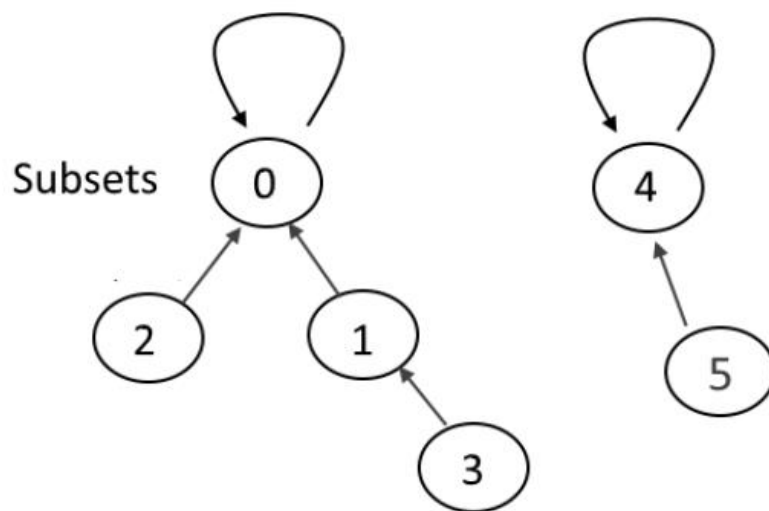
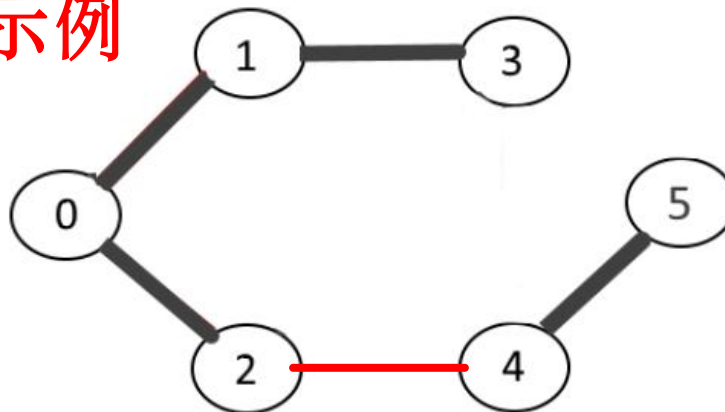


Kruskal算法(克鲁斯卡尔)



不相交集合数据结构-示例

如果现在还有一条边(2, 4)



Kruskal算法(克鲁斯卡尔)



不相交集数据结构—按秩合并

总是将更浅的树连接至更深的树上

➤ **function** MakeSet(x)

 x.parent := x

 x.rank := 0

➤ **function** Union(x, y)

 xRoot = Find(x); yRoot = Find(y)

 if xRoot == yRoot

 return

 if xRoot.rank < yRoot.rank

 xRoot.parent = yRoot

 else if xRoot.rank > yRoot.rank

 yRoot.parent = xRoot

 else

 yRoot.parent := xRoot

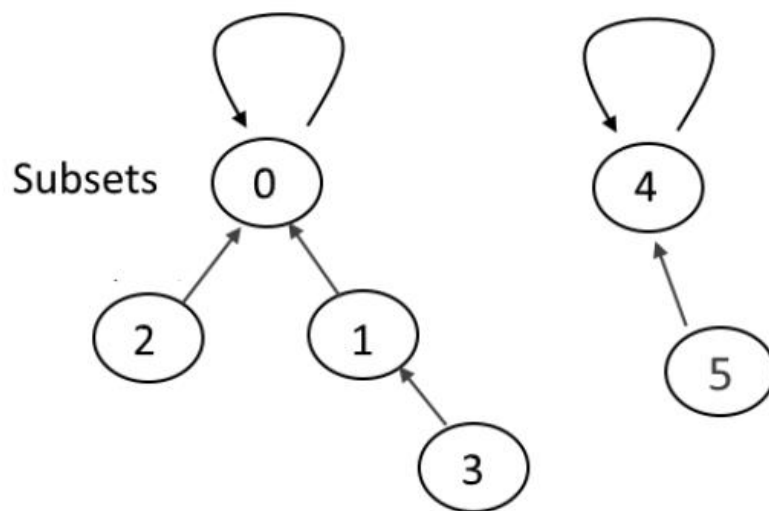
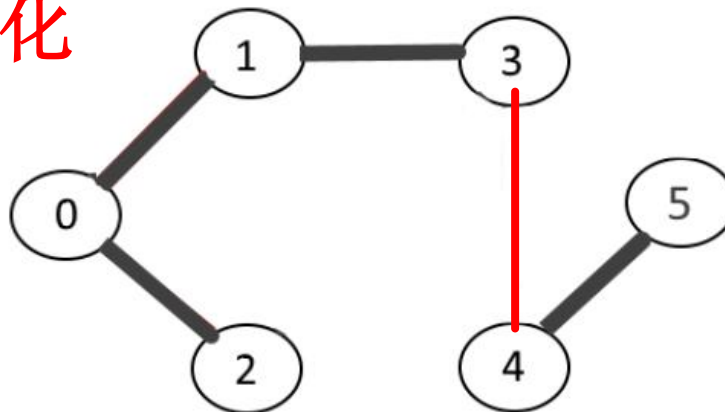
 xRoot.rank := xRoot.rank + 1

Kruskal算法(克鲁斯卡尔)



不相交集合数据结构优化

如果现在还有一条边(3, 4)



Kruskal算法(克鲁斯卡尔)



不相交集合数据结构—路径压缩

Find递归地经过树，改变每一个节点的引用到根节点

```
➤ function Find(x)
    if x.parent != x
        x.parent = Find(x.parent)
    return x.parent
```

Kruskal算法(克鲁斯卡尔)

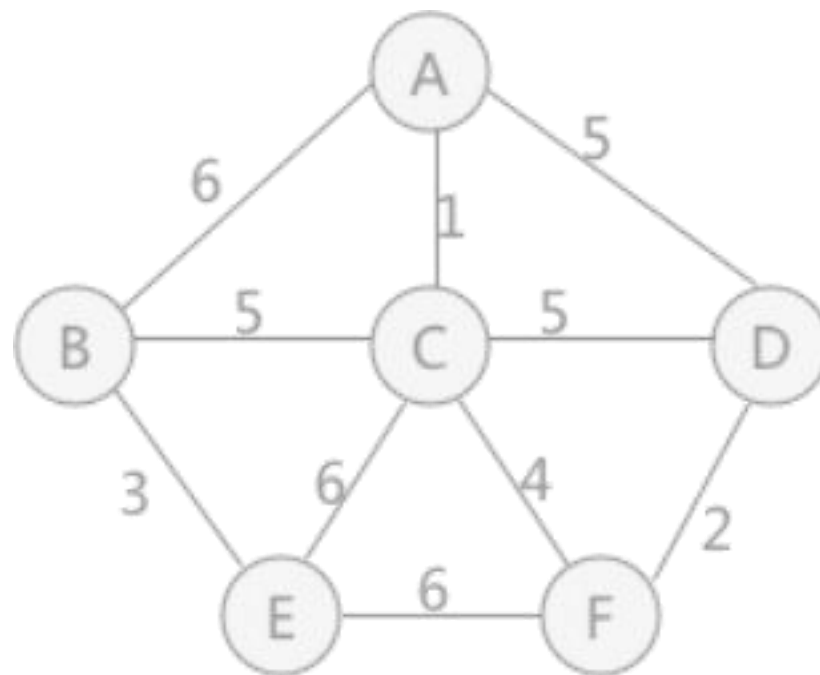


Kruskal算法伪代码

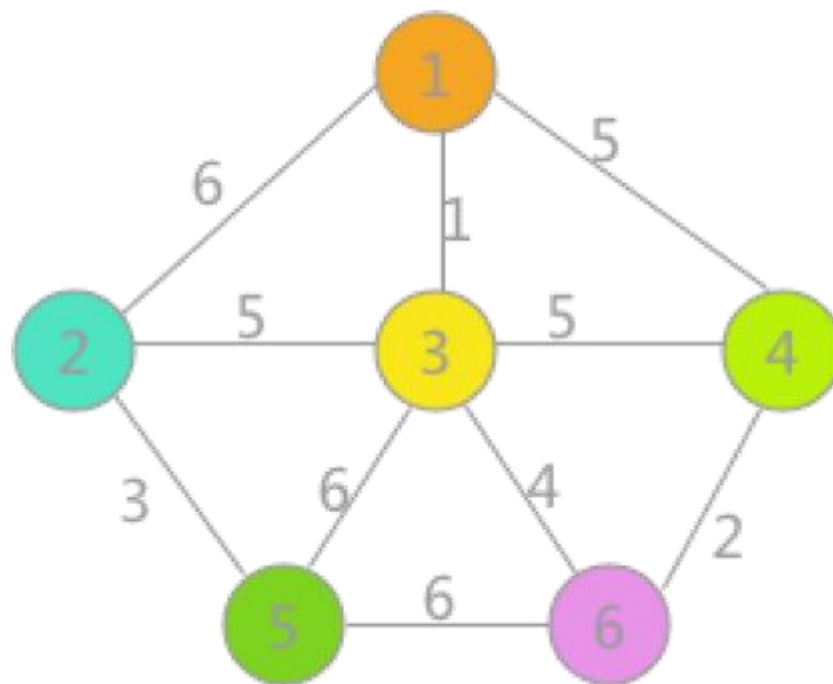
- Kruskal(G)
 $A = \emptyset$;
 for each $v \in V$
 MAKE-Tree(v)
 sort E ;
 for each $e = (u, v)$ in E in nondecreasing order: } $O(|E| \log |E|)$
 if FIND-ROOT(u) $\neq$ FIND-ROOT(v) } $O(\alpha(|V|))$
 $A = A \cup \{(u, v)\}$;
 UNION(u, v) } $O(|E|)$

于是，整体复杂度是 $O(|E| \log |E|) = O(|E| \log |V|)$

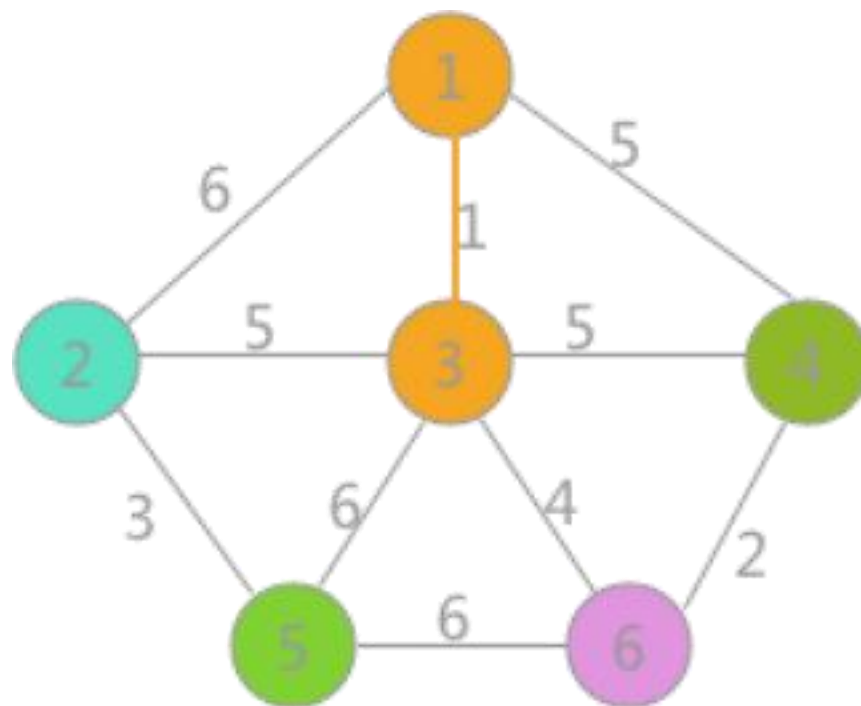
Kruskal算法(克鲁斯卡尔)



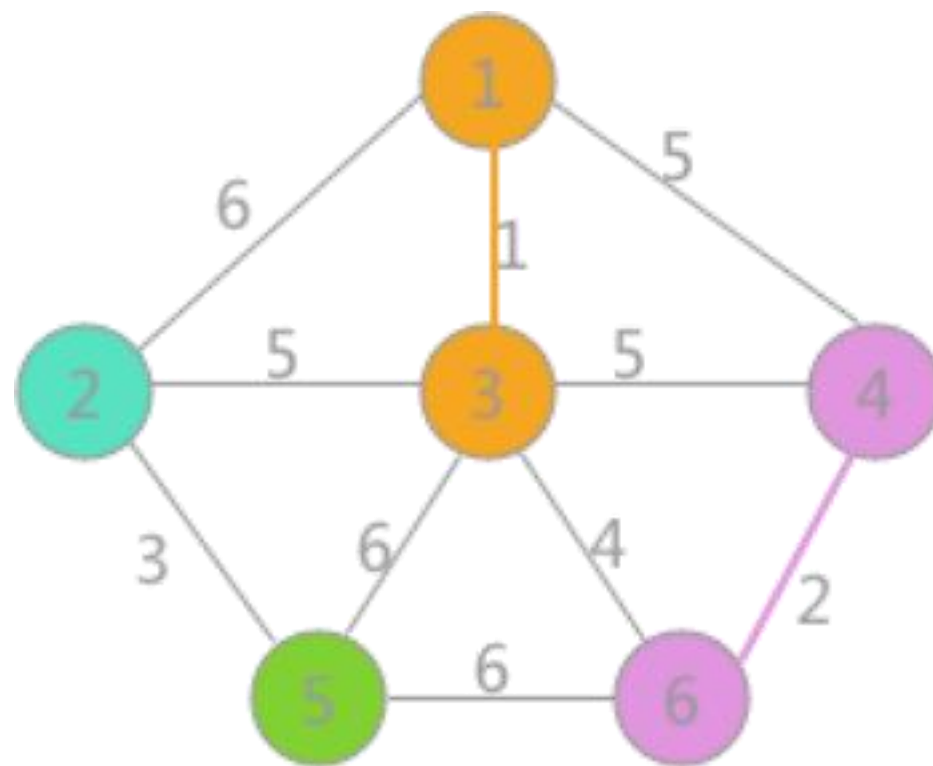
Kruskal算法(克鲁斯卡尔)



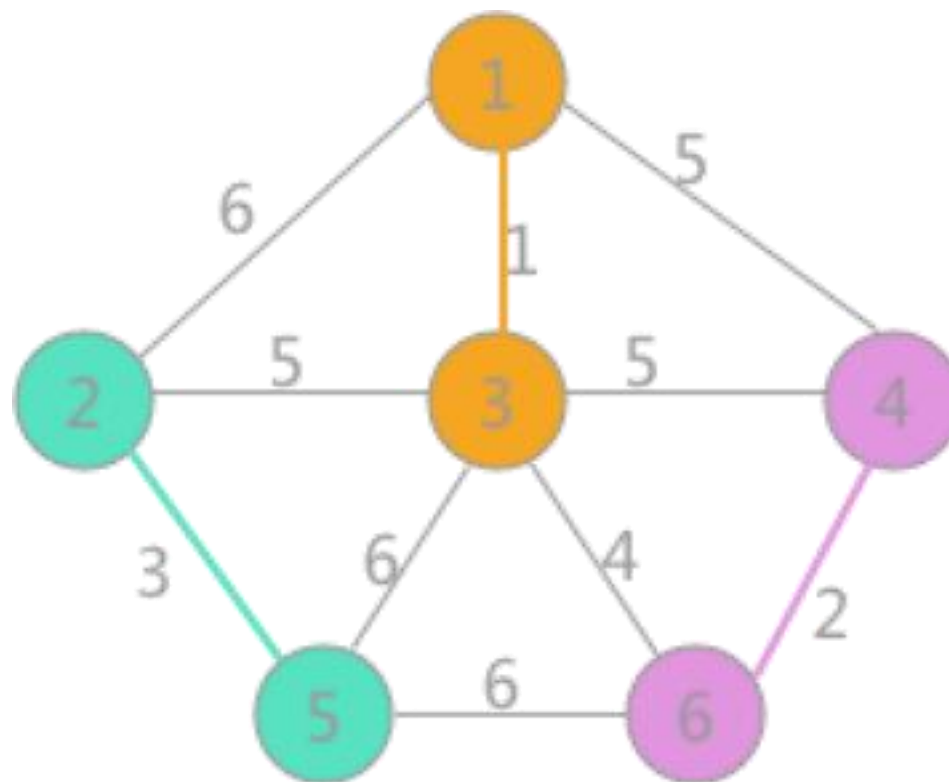
Kruskal算法(克鲁斯卡尔)



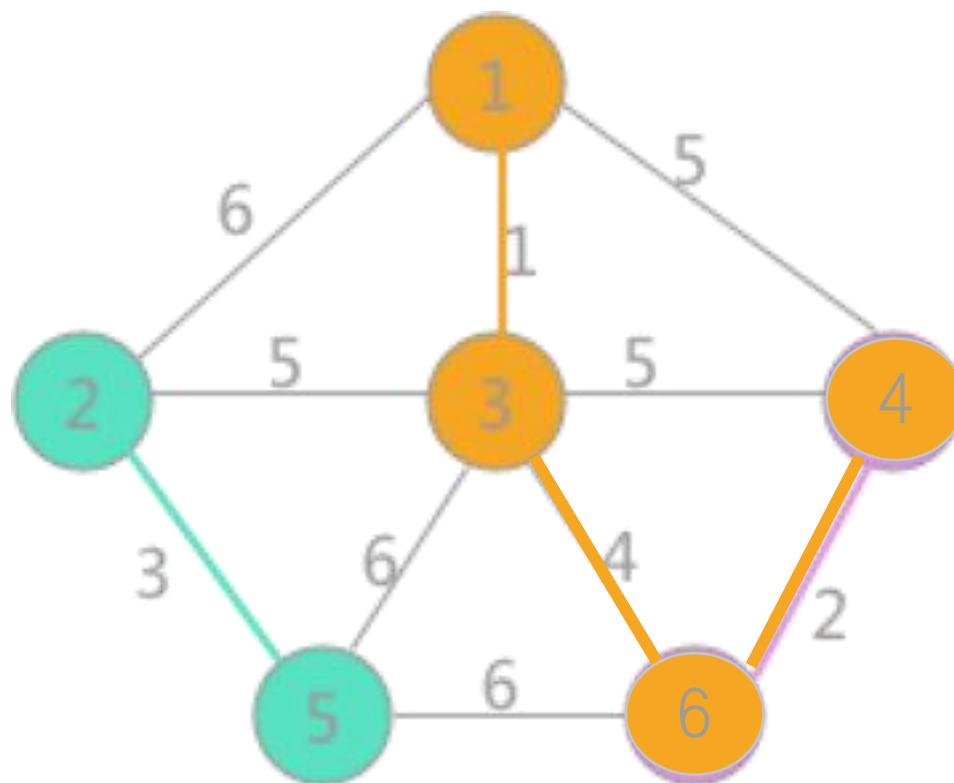
Kruskal算法(克鲁斯卡尔)



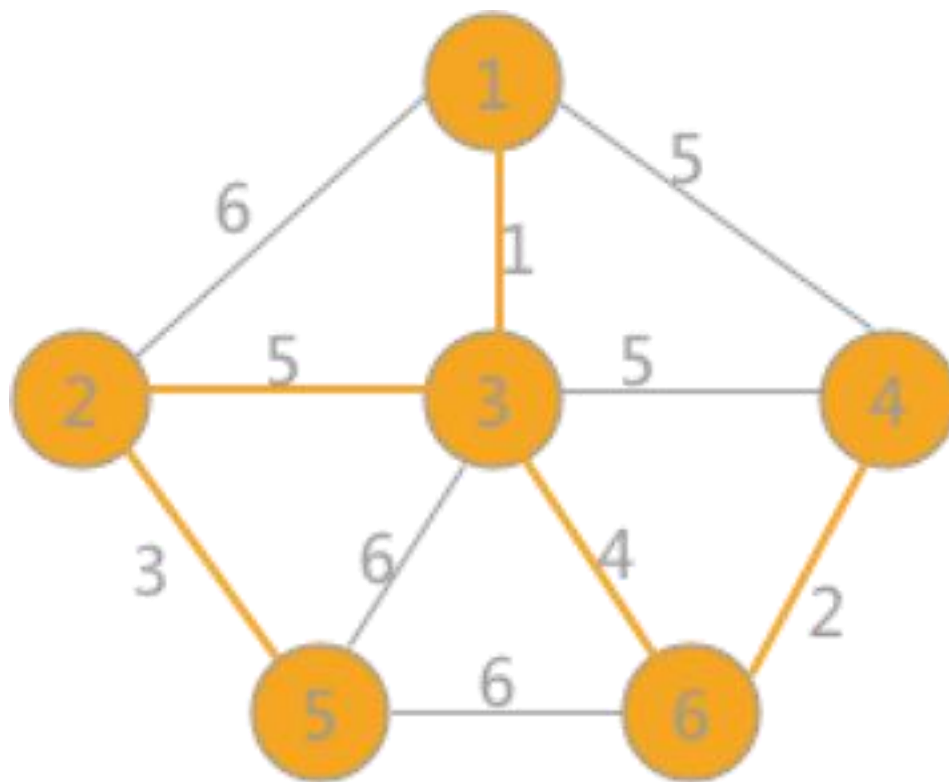
Kruskal算法(克鲁斯卡尔)



Kruskal算法(克鲁斯卡尔)



Kruskal算法(克鲁斯卡尔)



目录

第一节 最小生成树定义

第二节 Kruskal算法 (克鲁斯卡尔)

第三节 不相交集合应用

第四节 Prim算法 (普里姆)

第五节 斐波拉契堆



最小边属性切割

Peng Peng, M. Tamer Özsu, Lei Zou, Cen Yan, Chengjun Liu. MPC: Minimum Property-Cut RDF Graph Partitioning. ICDE 2022

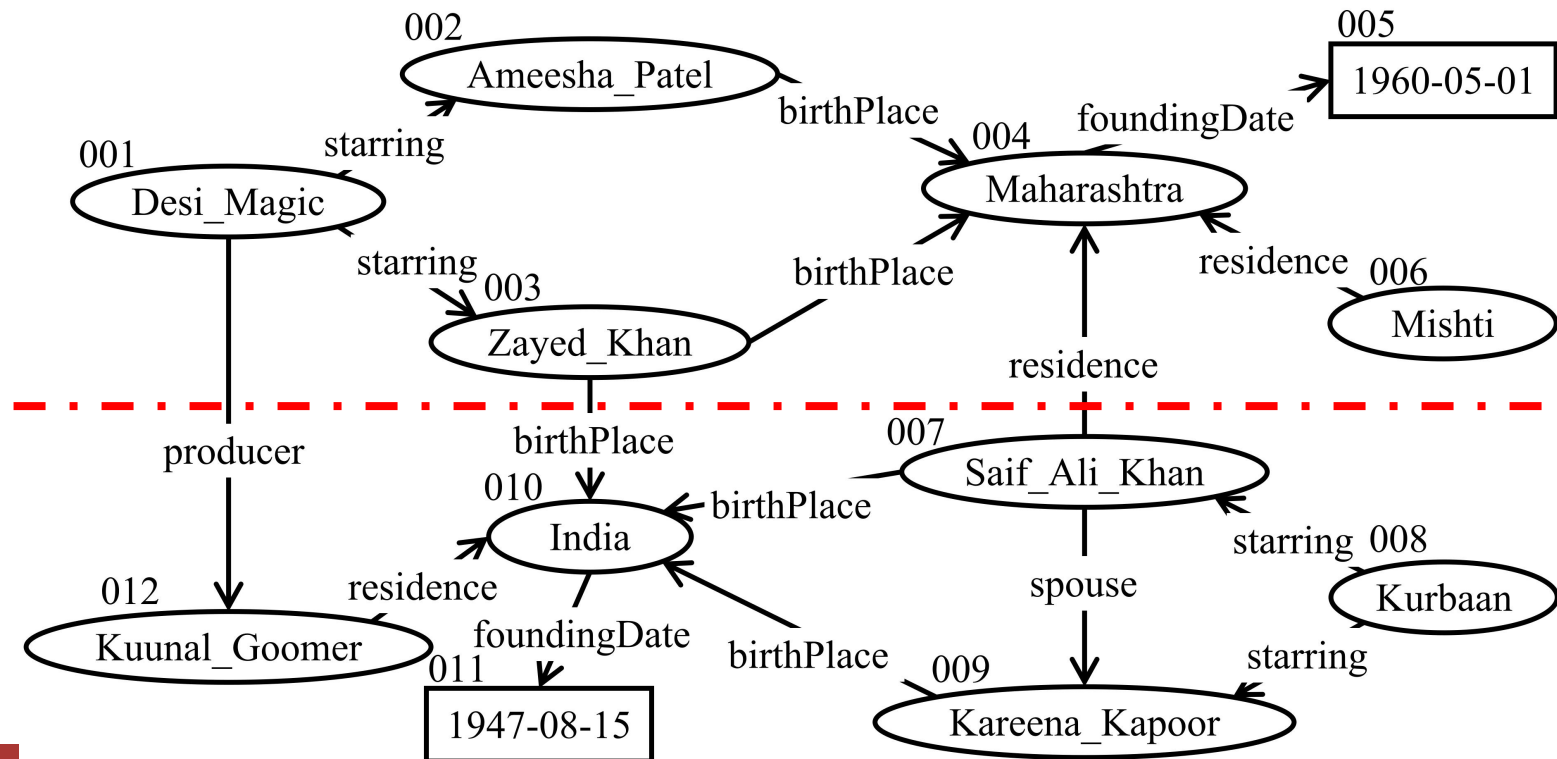


我们提出了一种新的点不相交的边标签图划分方案, Minimum Property-Cut (MPC, 最小属性划分), 用来提升分布式子图查询处理性能。

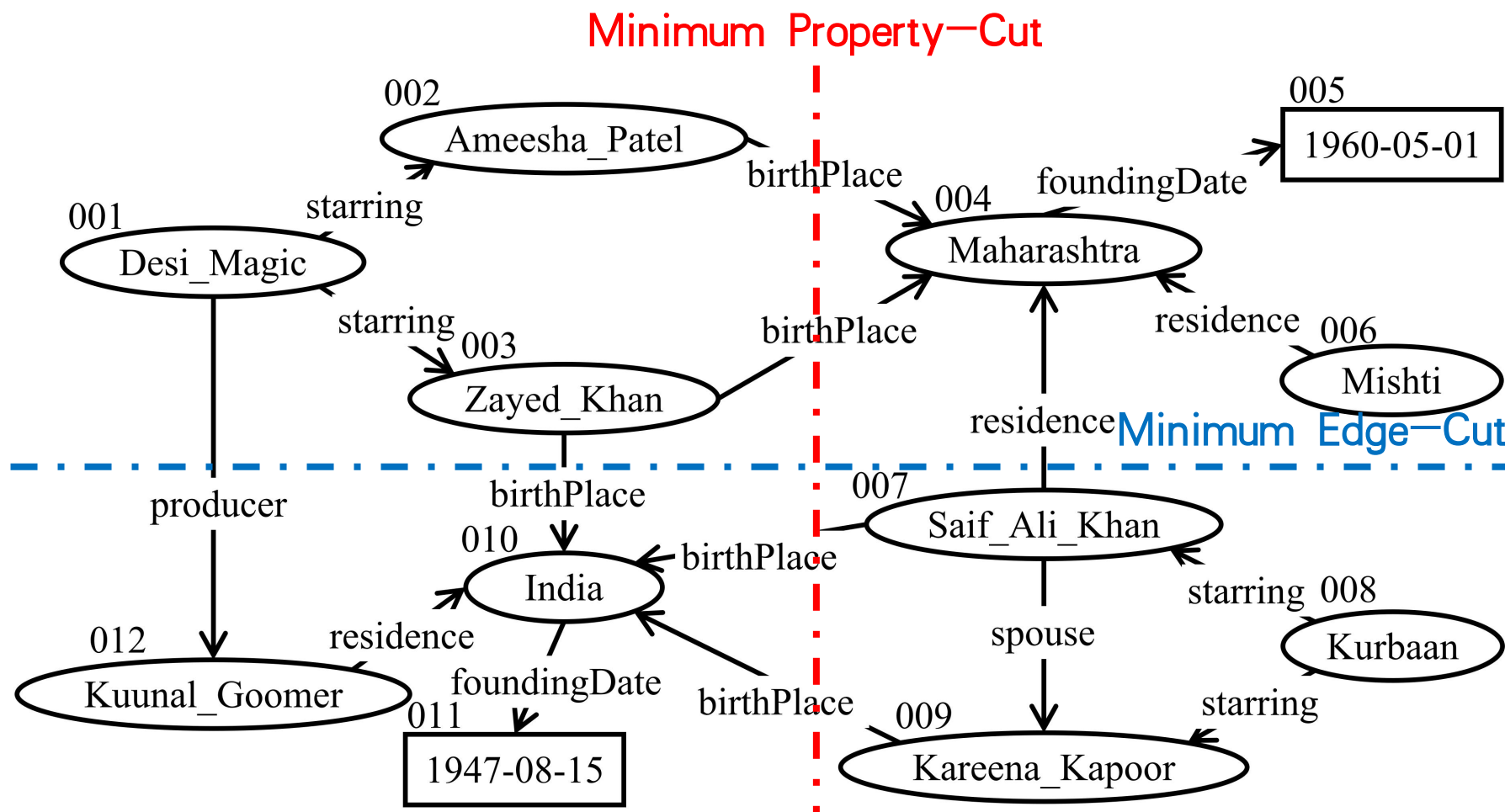
内部属性和跨界属性



- 当且仅当存在至少一条带有该属性的跨分区边，则该属性被称为**跨分区属性**，记为 L_{cross} ；否则，它被称为**内部属性**，记为 L_{in} 。



优化目标



问题定义



- 给定一个RDF图G和一个正整数k, G的Minimum Property-Cut (MPC) 划分 $F = \{F_1, F_2, \dots, F_k\}$ 满足以下条件:
 - 跨分区属性的数量 $|L_{\text{cross}}|$ 最小化 (即内部属性的数量 $|L_{\text{in}}|$ 最大化) ;
 - 对于每个 F_i , F_i 的大小 (即 $|V_i|$) 不大于 $(1 + \epsilon) \times |V|/k$, 其中 ϵ 是用户定义的分区的最大不平衡比例 (即分区的相对大小可以有多大差异) 。

Minimize Property-Cuts

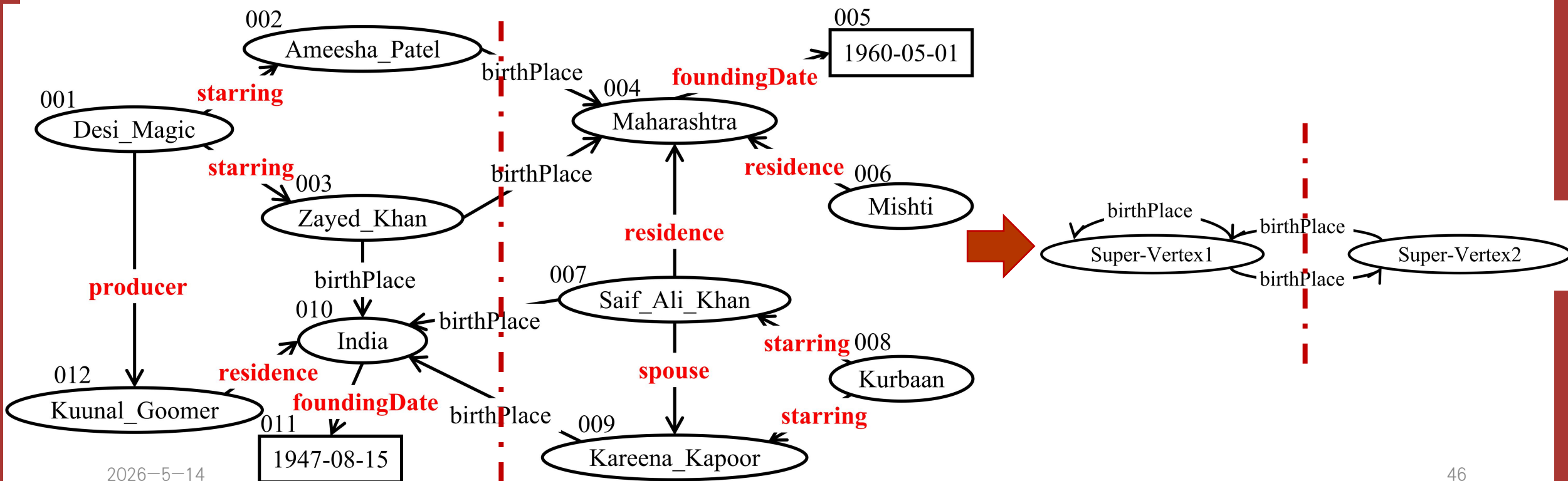


选择内部属性

粗化

划分粗化图

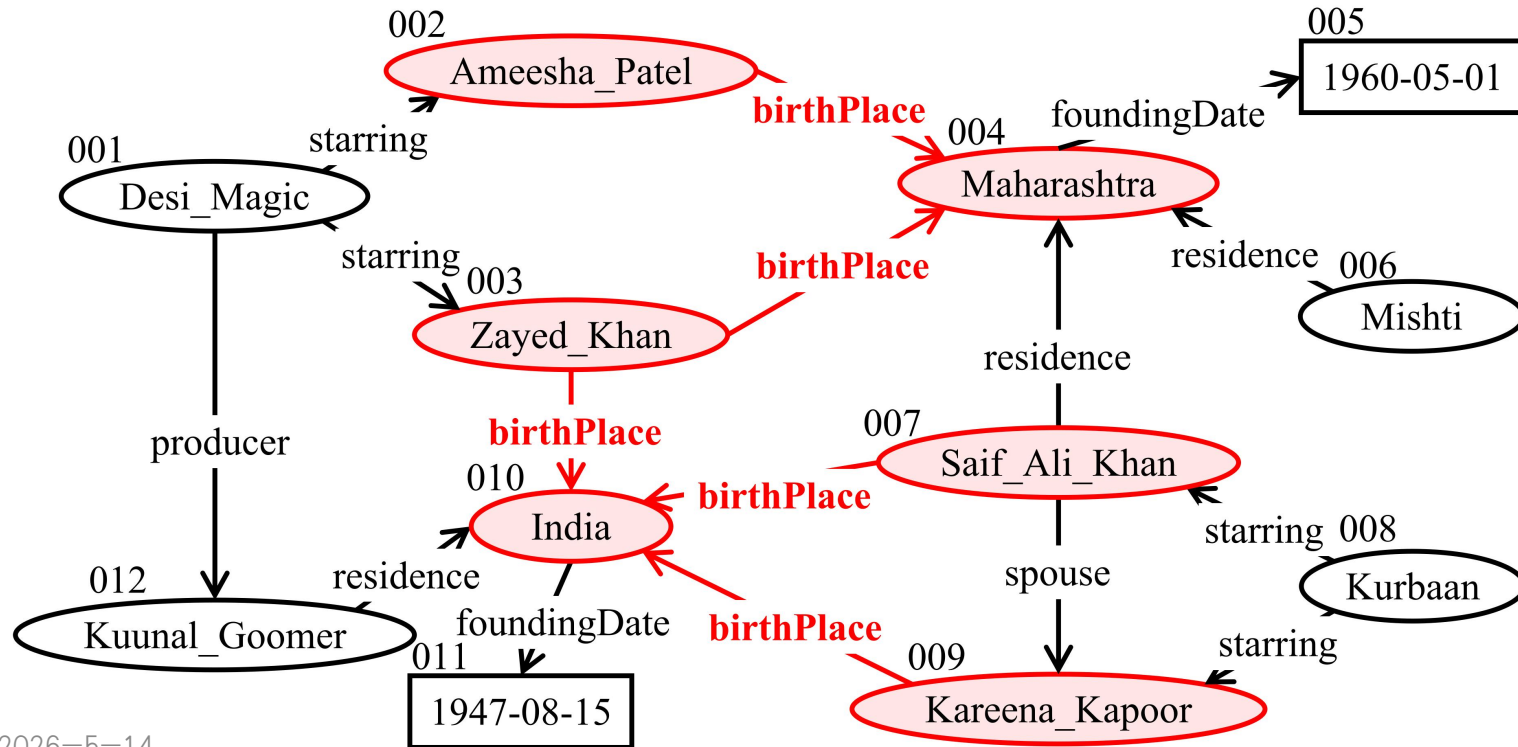
回溯



内部属性的限制



- 如果属性 p 是一个内部属性，那么在 p 的诱导图的弱连通分量中的任意两个顶点应该在同一个分区中。



如果 $birthPlace$ 是内部属性，6个红色点都应 在一个分区中

选择内部属性



给定一组属性 L' ，将它们选为内部属性的代价被定义为属性诱导子图 $G[L']$ 中最大弱连通分量的大小：

$$Cost(L') = \max_{c \in WCC(G[L'])} |c|$$

其中 c 是 $WCC(G[L'])$ 中的一个弱连通分量， $|c|$ 表示 c 中顶点的数量。

内部属性选择算法



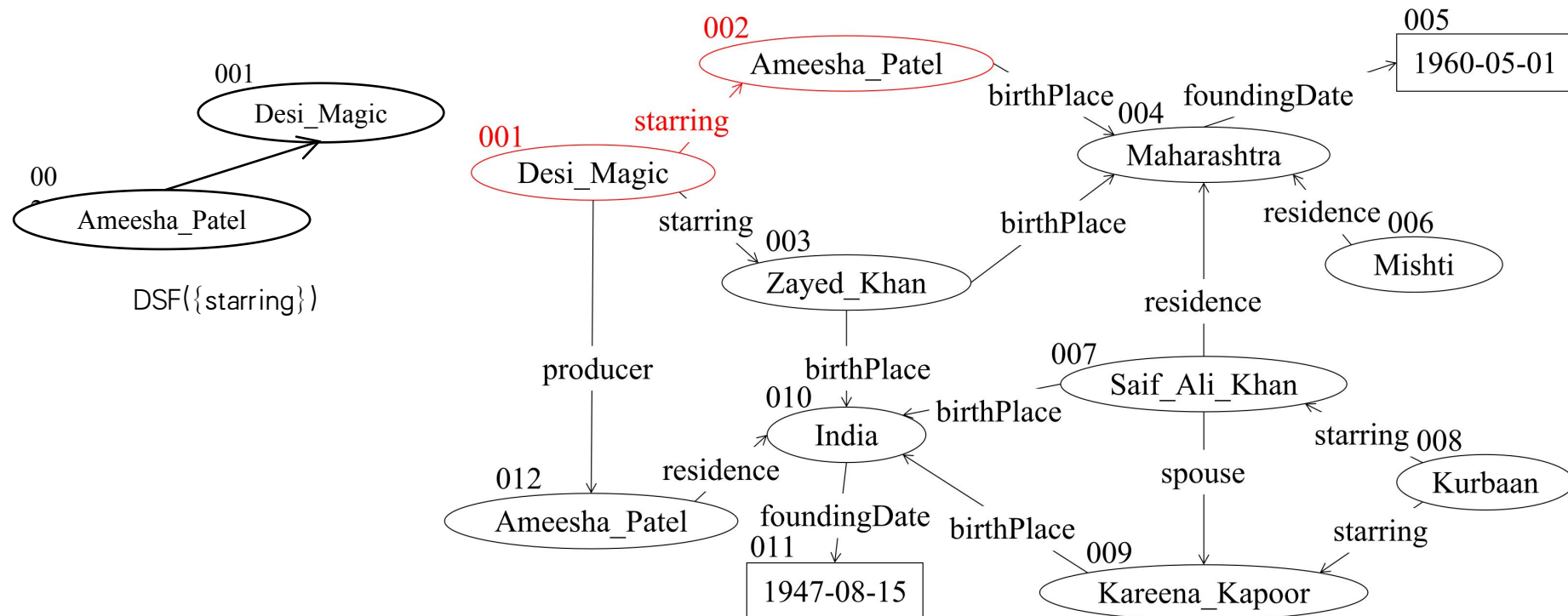
1. 首先，我们初始化一个空集合 L_{in} ，用于存储内部属性，并计算每个属性的属性诱导子图的弱连通分量。
2. 然后，我们迭代地选择一个属性 p ，该属性最小化 $Cost(L_{in} \cup \{p\})$ ，并将其插入到 L_{in} 中。这些步骤会重复进行，直到不能再选择更多的属性为止。

为了计算和合并弱连通分量，我们使用不相交集合森林数据结构（也就是并查集）。

基于不相交集数据结构的问题求解



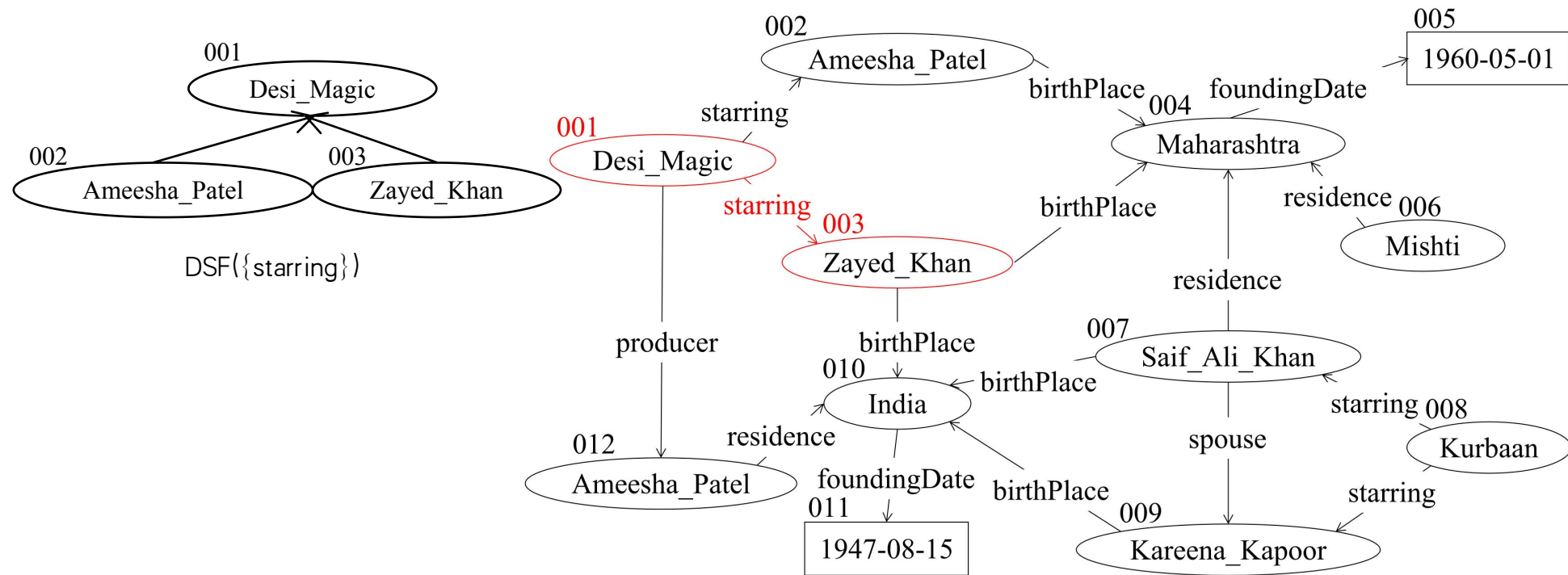
For edge $\langle 001, 002 \rangle$, we find the subsets of “starring” and union them by making 001 as the parent of 002 for starring



基于不相交集数据结构的问题求解



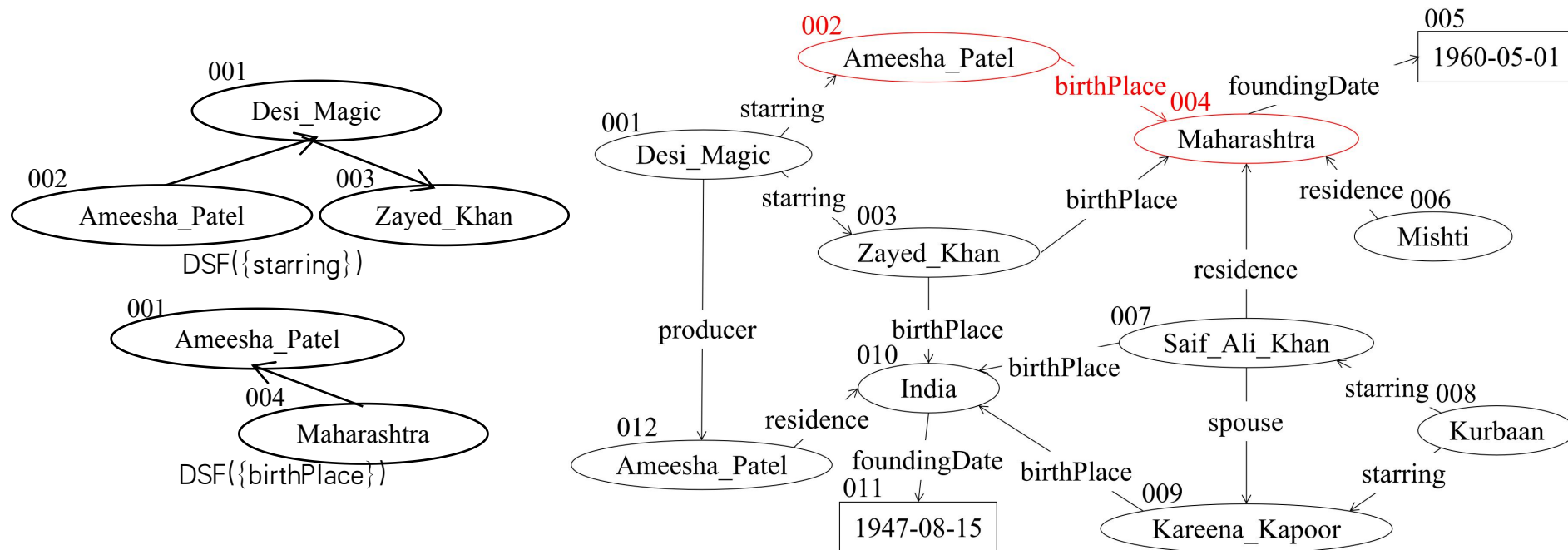
For edge $\langle 001, 003 \rangle$, we find the subsets of “starring” and union them by making 001 as the parent of 003 for starring



基于不相交集数据结构的问题求解



For $\langle 002, 004 \rangle$, we find the subsets of “birthPlace” and union them by making 002 as the parent of 004 for birthPlace



基于不相交集集合数据结构的问题求解

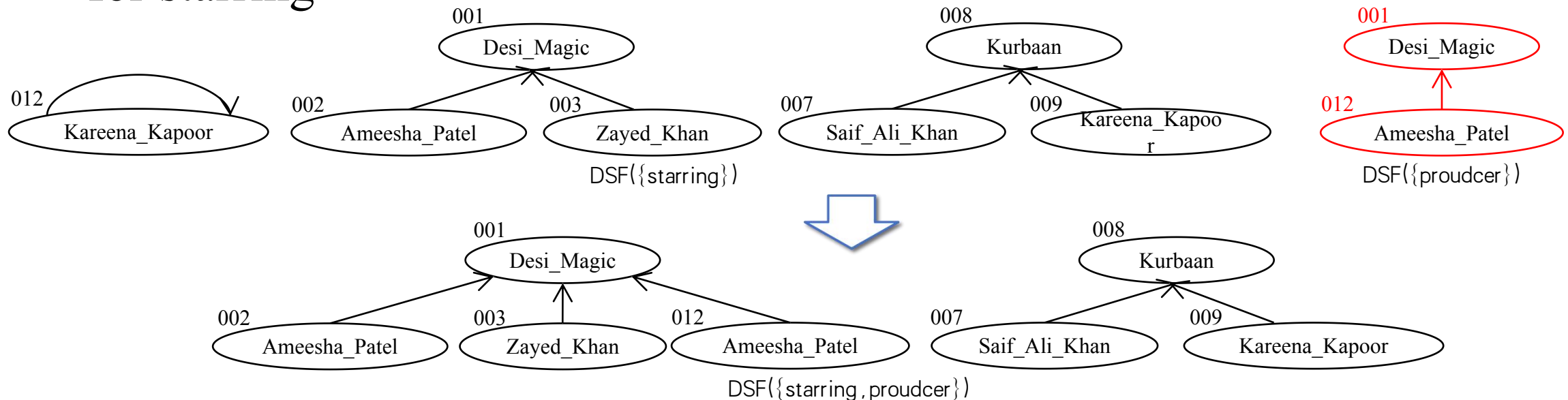


Two disjoint-set forests can join together by scanning one disjoint-set forest to union another one

基于不相交集数据结构的问题求解



For 012 and its set representative 001 in $DSF(\{\text{proudcer}\})$, we update the subsets of “starring” and union them by making 001 as the parent of 012 for starring



目录

第一节 最小生成树定义

第二节 Kruskal算法 (克鲁斯卡尔)

第三节 不相交集合应用

第四节 **Prim算法 (普里姆)**

第五节 斐波拉契堆

最小生成树定义



最优子结构性质

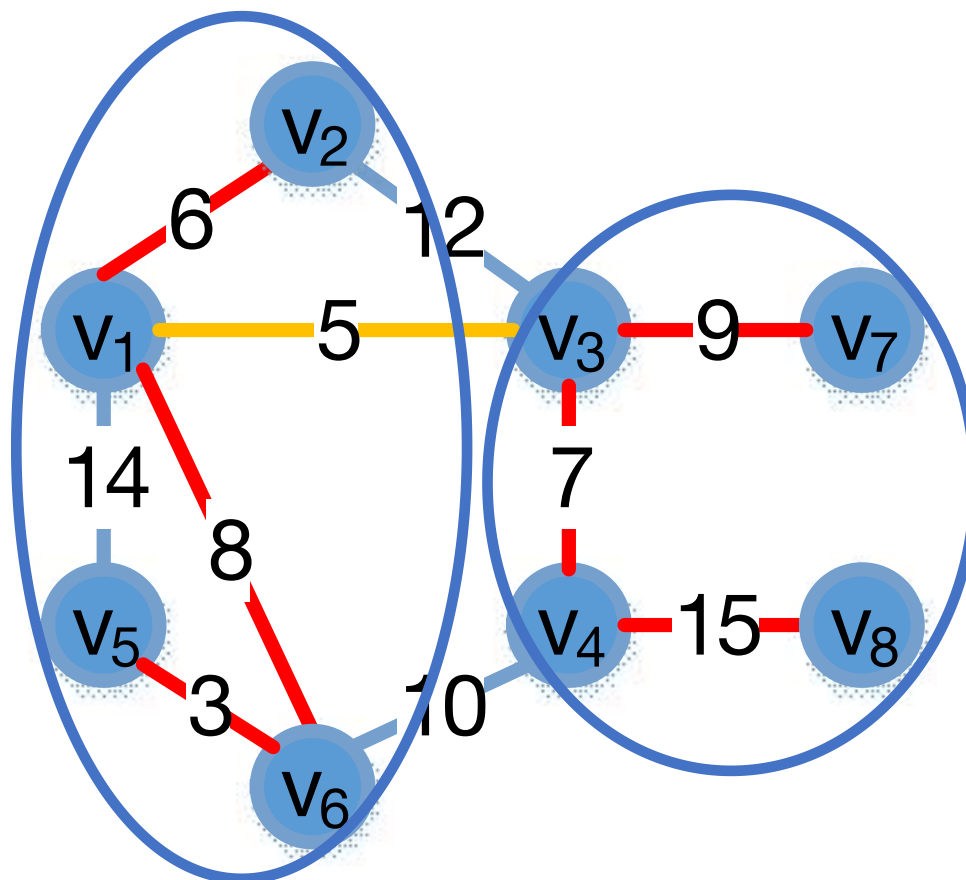
给定一个无向图 $G=(V, E, W)$ 以及它的最小生成树 T ，如果 $U \subseteq V$ 且边 (u,v) 是连接 U 和 $V-U$ 的所有边中权最小的边，那么边 (u,v) 在 T 中。

证明：反证法。如果 (u,v) 不在 T 中，那么设 T 中连接 U 和 $V-U$ 边是 (u^*,v^*) ，那么我们可以构造一个 T^* ，就是 $T+\{(u,v)\}-\{(u^*,v^*)\}$ ， T^* 权重比 T 还小。这个不可能的。

最小生成树定义



最优子结构示例



Prim算法(普里姆)

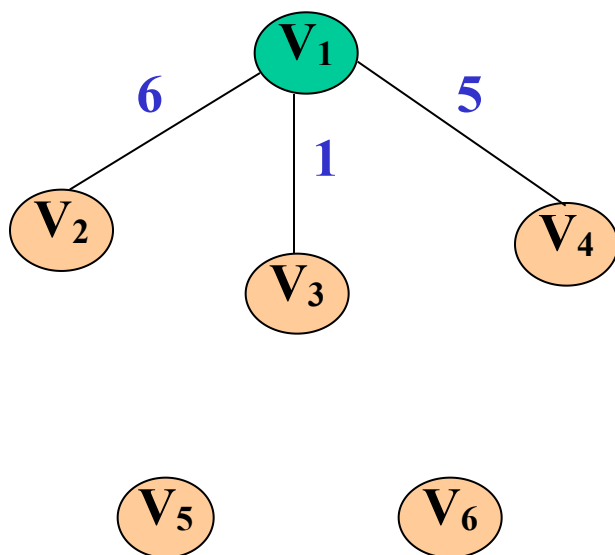
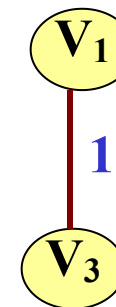


- 对于图 $G=(V, E, W)$, TE 是 N 上最小生成树中边的集合。
- 算法从 $U=\{u_0\}(u_0 \in V)$, $TE=\{\}$ 开始, 重复执行下述操作:
 - ☞ 在所有 $u \in U$, $v \in V-U$ 的边 (u, v) 中找一条代价最小的边 (u_0, v_0) , 将其并入集合 TE , 同时将 v_0 并入 U 集合。
 - ☞ 当 $U=V$ 则结束, 此时 TE 中必有 $n-1$ 条边, 则 $T=(V, \{TE\})$ 为 G 的最小生成树。
- 普里姆算法构造最小生成树的过程是从一个顶点 $U=\{u_0\}$ 作初态, 不断寻找与 U 中顶点相邻且代价最小的边的另一个顶点, 扩充到 U 集合直至 $U=V$ 为止。

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

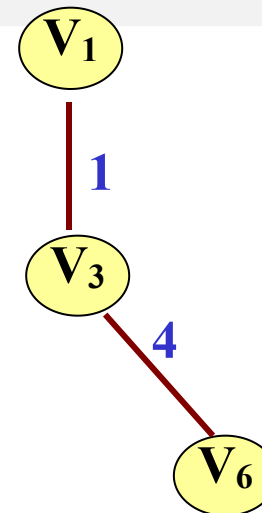
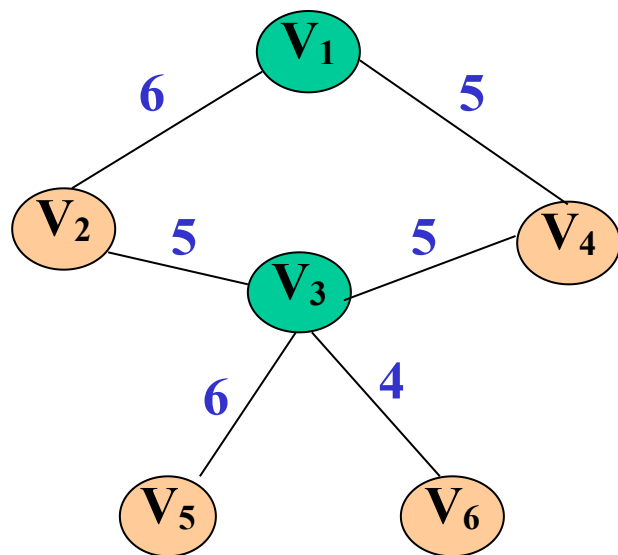


步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

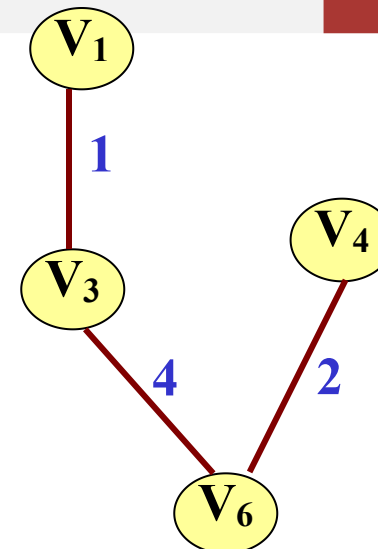
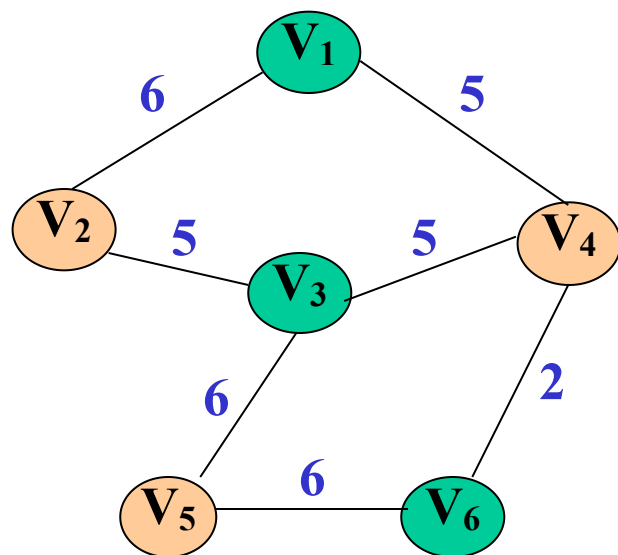


步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{V ₁ , V ₃ , V ₆ }	{V ₂ , V ₄ , V ₅ }

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

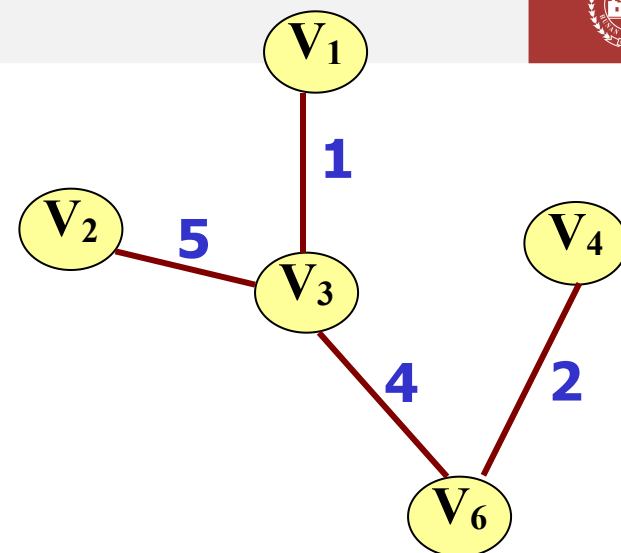
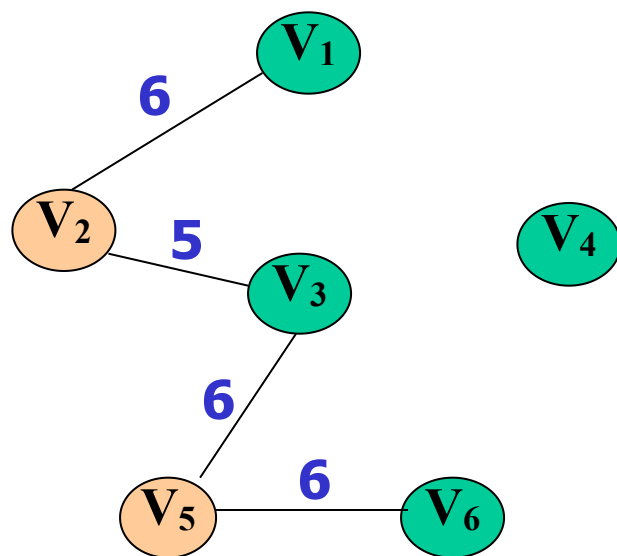


步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{V ₁ , V ₃ , V ₆ }	{V ₂ , V ₄ , V ₅ }
(3)	{V ₁ , V ₃ , V ₆ , V ₄ }	{V ₂ , V ₅ }

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

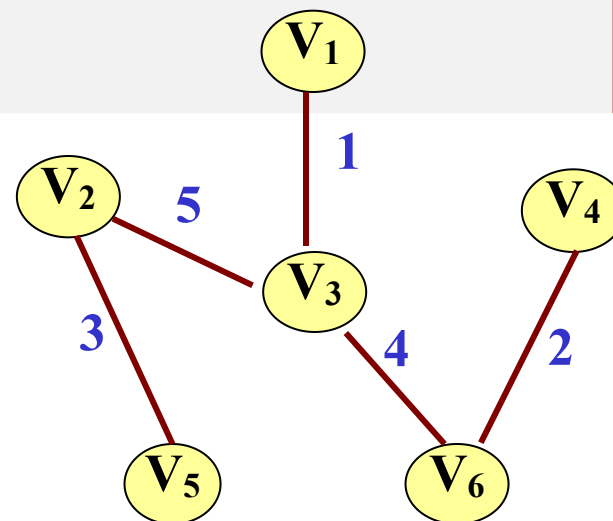
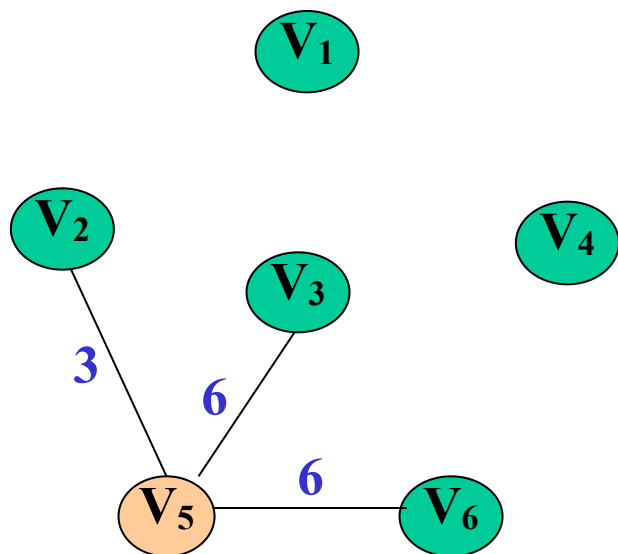


步骤	U	V-U
(0)	{ V ₁ }	{ V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{ V ₁ , V ₃ }	{ V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{ V ₁ , V ₃ , V ₆ }	{ V ₂ , V ₄ , V ₅ }
(3)	{ V ₁ , V ₃ , V ₆ , V ₄ }	{ V ₂ , V ₅ }
(4)	{ V ₁ , V ₃ , V ₆ , V ₄ , V ₂ }	{ V ₅ }

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

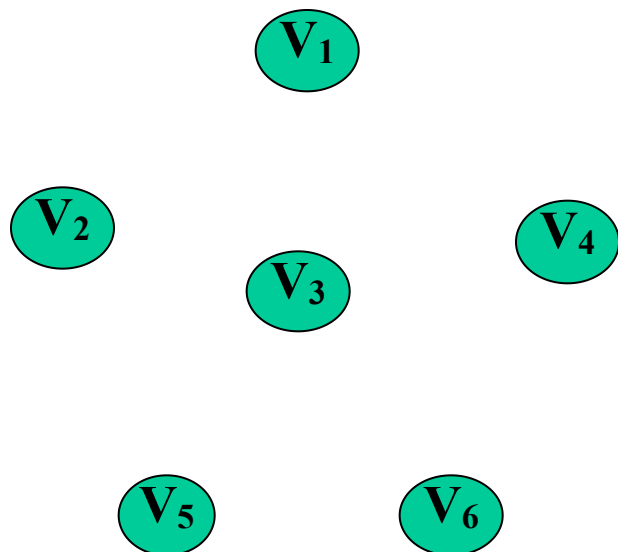
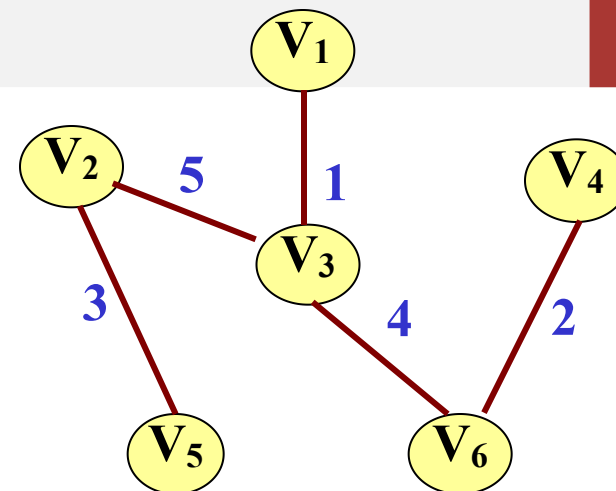


步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{V ₁ , V ₃ , V ₆ }	{V ₂ , V ₄ , V ₅ }
(3)	{V ₁ , V ₃ , V ₆ , V ₄ }	{V ₂ , V ₅ }
(4)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ }	{V ₅ }
(5)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ , V ₅ }	{ }

Prim算法(普里姆)



- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止

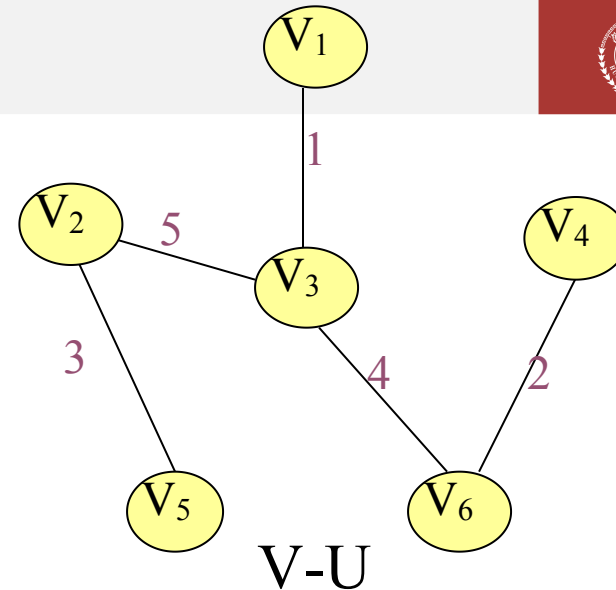
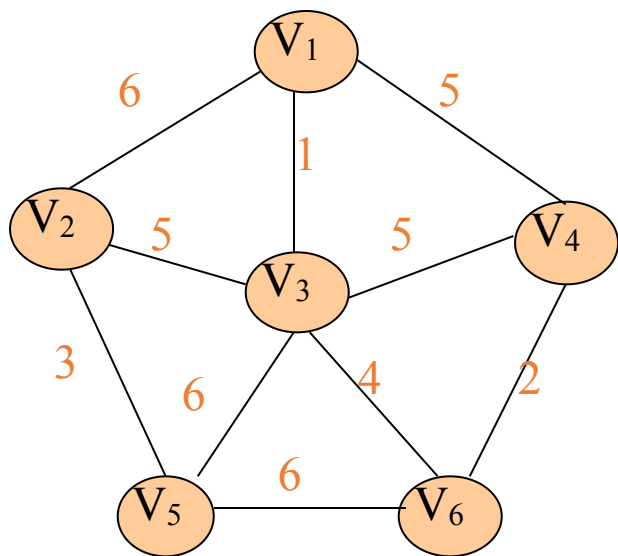


步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{V ₁ , V ₃ , V ₆ }	{V ₂ , V ₄ , V ₅ }
(3)	{V ₁ , V ₃ , V ₆ , V ₄ }	{V ₂ , V ₅ }
(4)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ }	{V ₅ }
(5)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ , V ₅ }	{ }

Prim算法(普里姆)

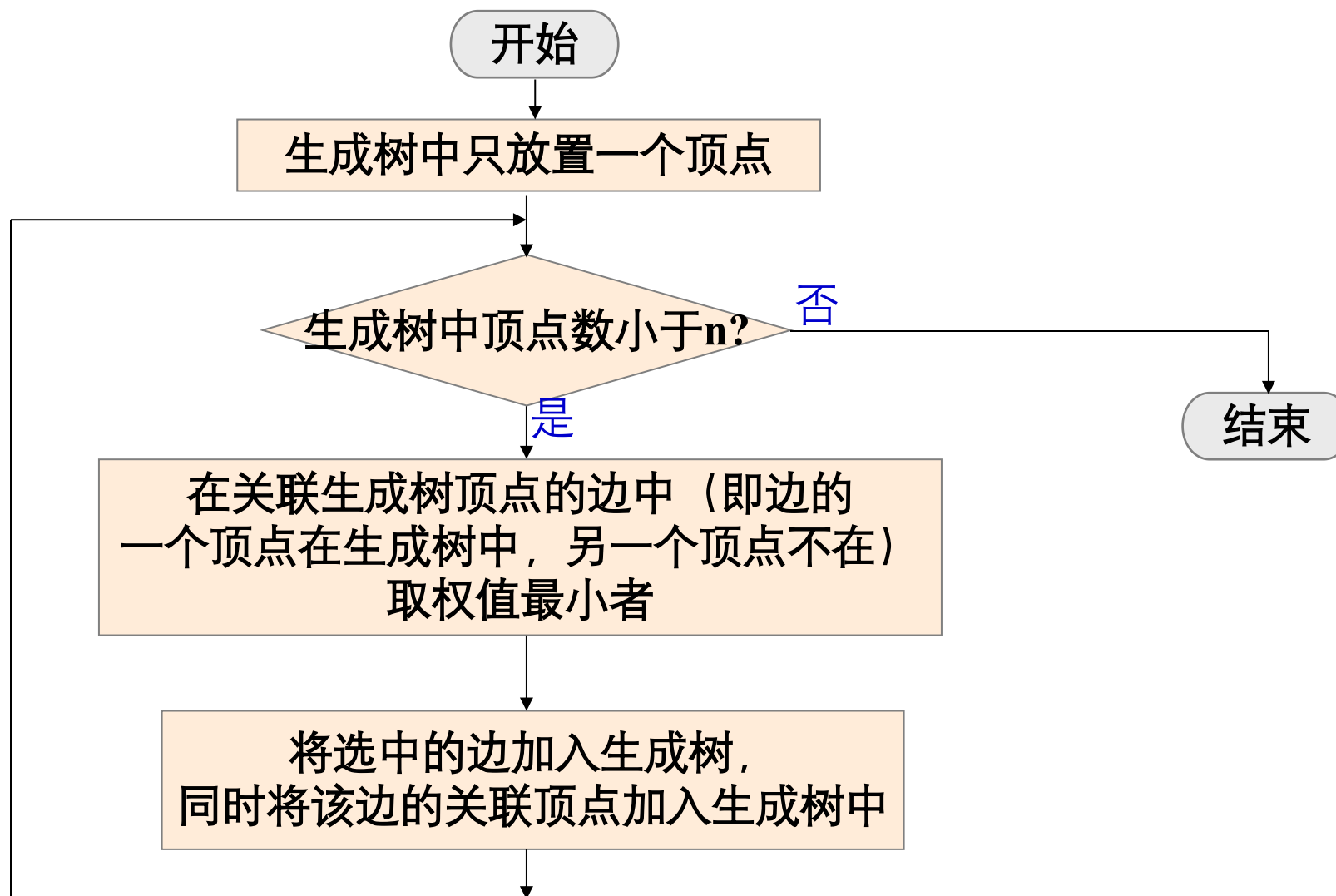


- 普里姆算法求最小生成树：从生成树中只有一个顶点开始，到顶点全部进入生成树为止。



步骤	U	V-U
(0)	{V ₁ }	{V ₂ , V ₃ , V ₄ , V ₅ , V ₆ }
(1)	{V ₁ , V ₃ }	{V ₂ , V ₄ , V ₅ , V ₆ }
(2)	{V ₁ , V ₃ , V ₆ }	{V ₂ , V ₄ , V ₅ }
(3)	{V ₁ , V ₃ , V ₆ , V ₄ }	{V ₂ , V ₅ }
(4)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ }	{V ₅ }
(5)	{V ₁ , V ₃ , V ₆ , V ₄ , V ₂ , V ₅ }	{ }

Prim算法(普里姆)



Prim算法(普里姆)



```
1 Prim(G, Adj, s)
2   key[s]=0;
3   for each  $v \in V - \{s\}$ 
4     do key[v] =  $\infty$ 
5   Q = V
6   while  $Q \neq \emptyset$ :
7     u = EXTRACT-MIN(Q);
8     for each  $v \in \text{Adj}[u]$ :
9       if key[v] > w(u, v)
10        then key[v] = w(u, v)
```

Prim算法(普里姆)

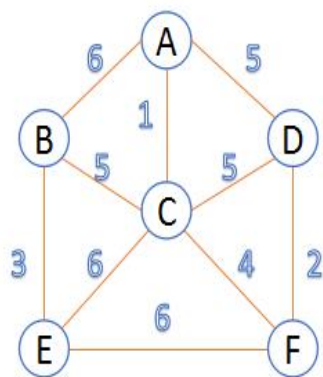


时间复杂度

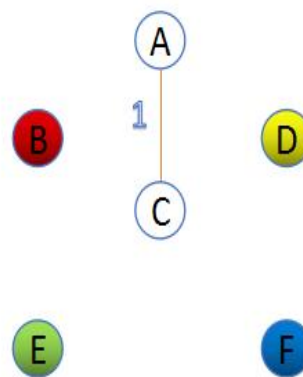
$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

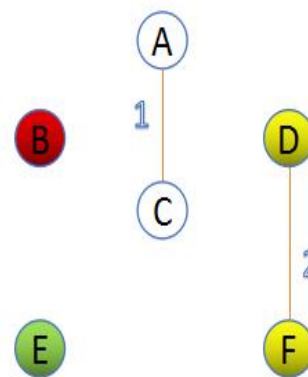
Kruskal算法与Prim算法对比



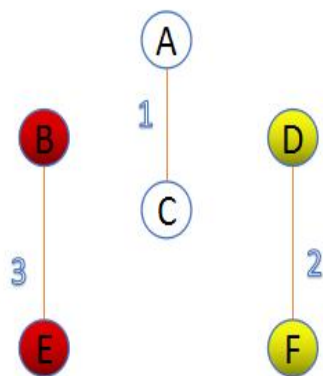
连通网G



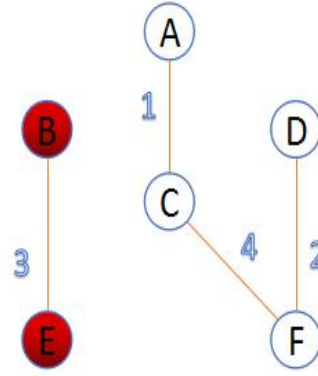
1. 选择代价最小的边(A,C);并保证A,C不在同一颗树上,然后合并A,C



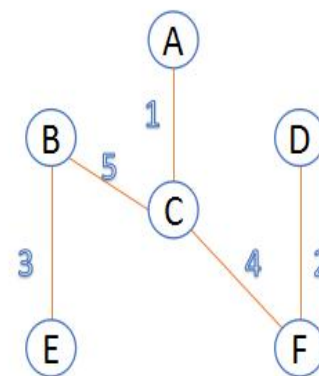
2. 选择代价最小的边(D,F);并保证D,F不在同一颗树上,然后合并D,F



3. 选择代价最小的边(B,E);并保证B,E不在同一颗树上,然后合并B,E

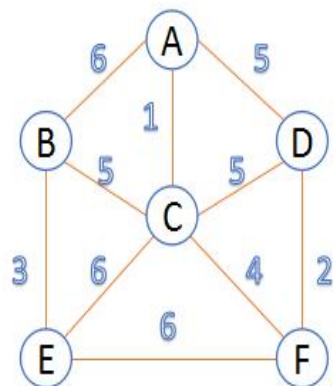


4. 选择代价最小的边(C,F);并保证C,F不在同一颗树上,然后合并C,F所在的树

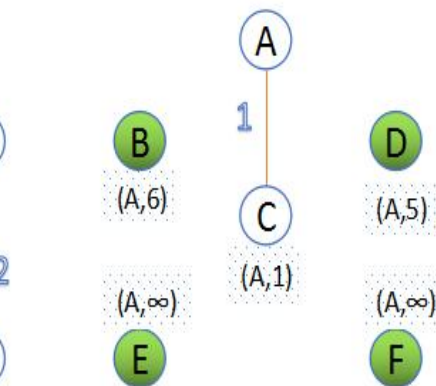


5. 选择代价最小的边(A,D),顶点A,D在同一颗树上,丢弃;
选择最小的边(C,D),顶点C,D在同一颗树上,丢弃;
选择最小的边(B,C),顶点B,C不在同一颗树上,加入此边,然后合并B,C所在的树,此时所有顶点在同一颗树上,返回;

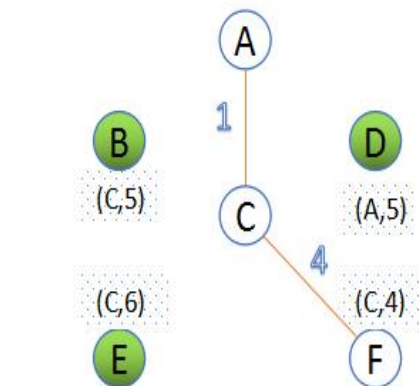
Kruskal算法与Prim算法对比



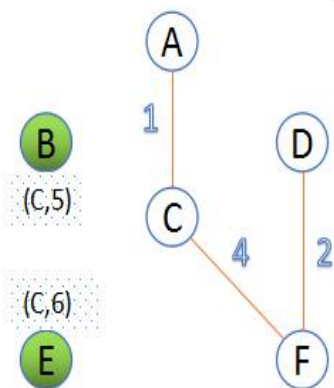
连通网G



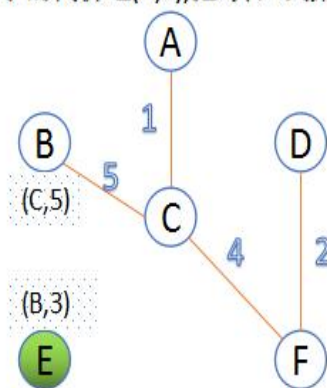
1. 初始 $u=\{A\}$, $v=\{B,C,D,E,F\}$; 顶点B下方(A,6), 表示与集合u中A的代价为6作为最小代价边。选择最小的代价边(A,C), 把C并入到集合u中。



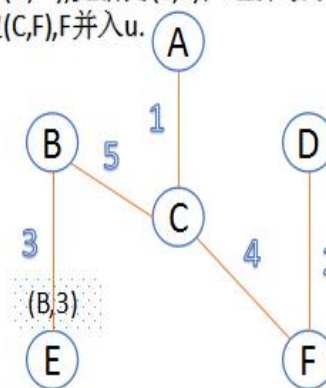
2. $u=\{A,C\}$, $v=\{B,D,E,F\}$; 更新v中顶点与集合u的最小的代价边; 例如: 顶点E之前为(A,∞), 更新为(C,6); 选择最小代价边(C,F), F并入u。



3. $u=\{A,C,F\}$, $v=\{B,D,E\}$; 更新v中顶点与集合u的最小的代价边; 选择最小代价边(F,D), D并入u。



4. $u=\{A,C,F,D\}$, $v=\{B,E\}$; 更新v中顶点与集合u的最小的代价边; 选择最小代价边(C,B), B并入u。



5. $u=\{A,C,F,D,B\}$, $v=\{E\}$; 更新v中顶点与集合u的最小的代价边; 选择最小代价边(B,E), E并入u。

目录

第一节 最小生成树定义

第二节 Kruskal算法 (克鲁斯卡尔)

第三节 不相交集合应用

第四节 Prim算法 (普里姆)

第五节 斐波拉契堆

回顾堆的定义



堆实质上是满足如下性质的完全二叉树：树中任一非叶结点的关键字均不大于(或不小于)其左右孩子(若存在)结点的关键字

对应序列上， n 个元素的序列 $\{k_1, k_2, \dots, k_n\}$ 当且仅当满足如下关系时，称之为**堆 (heap)**。若满足条件 $k_i \leq k_{i/2}$ 且则称**最大堆**；若满足条件 $k_i \geq k_{i/2}$ 则称**最小堆**

可合并堆



可合并堆 (Mergeable Heap) 是一种支持以下核心操作的抽象数据结构:

- ❑ **MAKE-HEAP()**: 创建空堆
- ❑ **INSERT(H, x)**: 插入元素
- ❑ **MINIMUM(H)**: 返回最小关键字 (或最大, 取决于堆类型)
- ❑ **EXTRACT-MIN(H)**: 删除并返回最小关键字
- ❑ **UNION(H₁, H₂)**: 将两个堆合并为一个, 并销毁原堆

在可合并堆的基础上, Prim算法还要求支持**DECREASE-KEY(H, x, k)**

(也就是将堆H中元素x赋值一个新的值k) 以及**DELETE(H, x)** (删除H中元素x) 。

二项堆



二项堆 (Binomial Heap) 其实是一种基于二项树构建的可合并堆数据结构。

二项树



- **二项树**是构建二项堆的基本单元，其结构呈递归形式，通常用 B_k 来表示度为 k 的二项树。
 - B_0 仅包含一个节点。
 - B_k ($k > 0$) 由两棵 B_{k-1} 树连接而成，其中一棵的根成为另一棵根的最左子节点。

二项树的形状



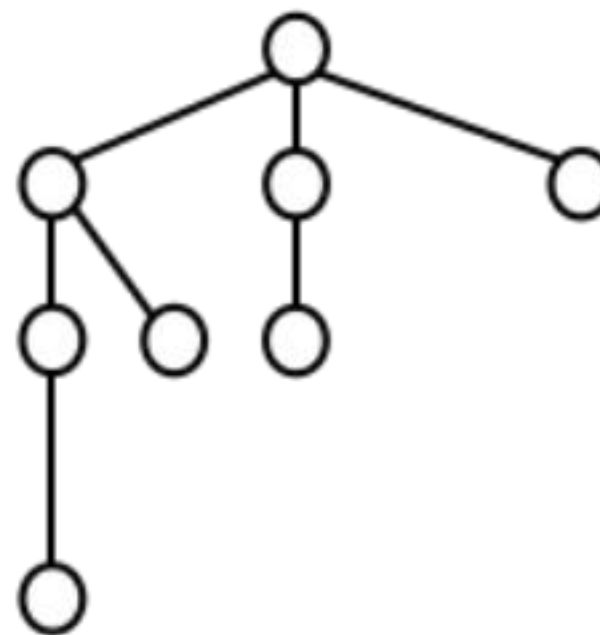
B_0



B_1



B_2



B_3

二项树性质



二项树具有节点数、高度、度数等方面的独特数学性质。

01

节点数量

一棵度为 k 的二项树 B_k 恰好包含 2^k 个节点。

02

树的高度

二项树 B_k 的高度为 k 。

03

根的度数

根节点的度数（子节点数量）为 k ，且其度数大于任何子节点的度数。

二项堆核心性质

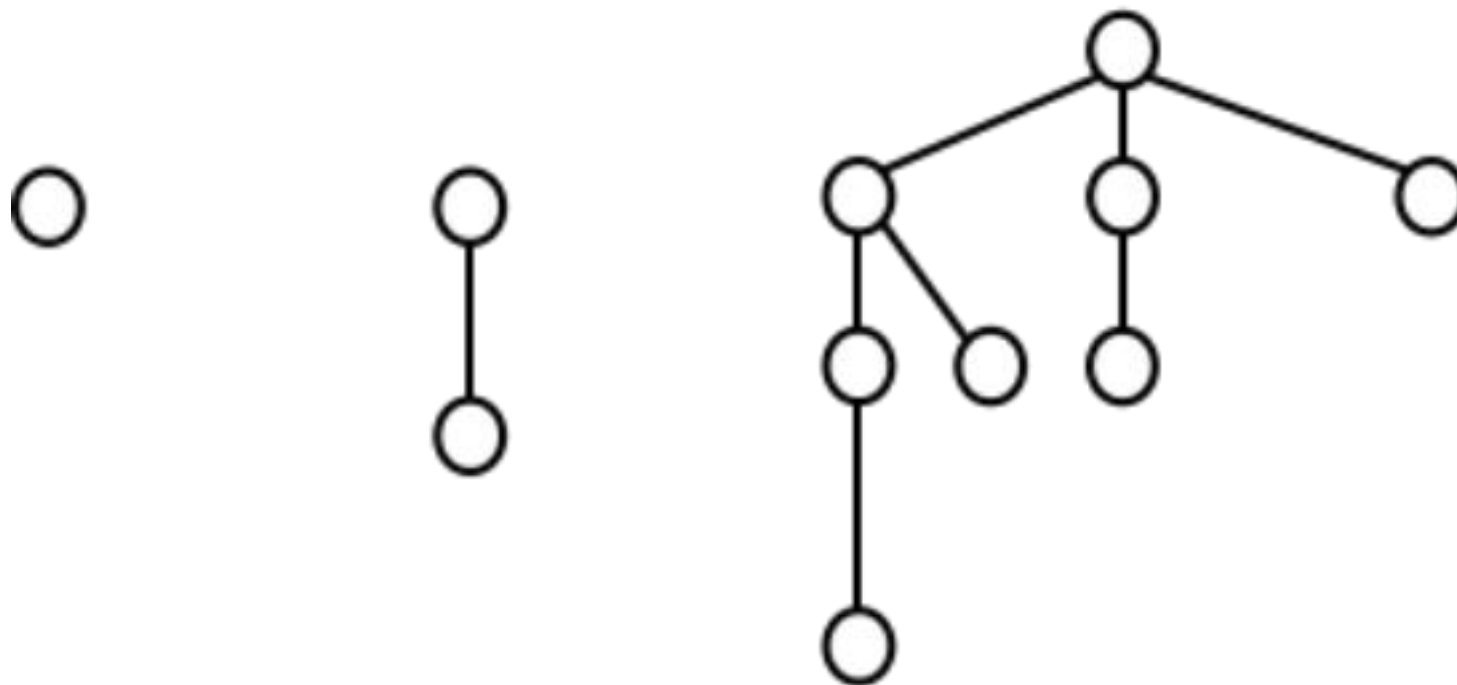


- 二项堆中二项树都是满足**最小堆性质**的，也就是二项树中每个元素比其孩子都小。
- 对于任意非负整数 k ，在同一个二项堆中，**最多只能存在一棵**度数为 k 的二项树 B_k 。

二项堆示例



□ 包含11个元素的堆



二项堆核心性质



□ 给定 n 个元素，二项树数量为 $O(\log n)$ ，二项树深度也为 $O(\log n)$

二项堆上操作——UNION

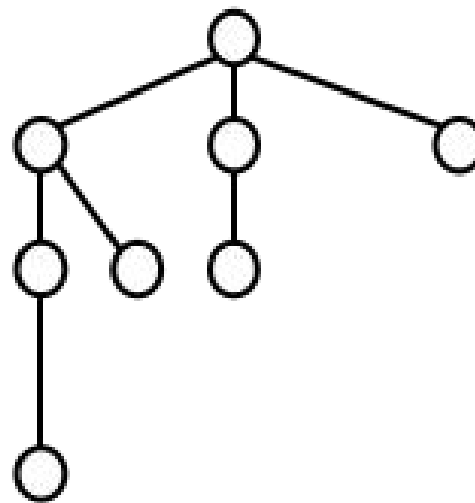
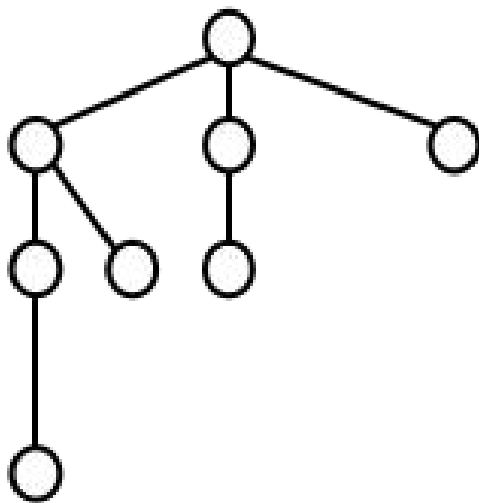


- 首先，我们将两个待合并的二项堆 H_1 和 H_2 中的所有二项树，按照它们的度数 k 从小到大进行排列。然后，像二进制加法一样“进位”合并相同度数的树。
- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有树，按阶排序（从小到大）。

二项堆上操作——UNION



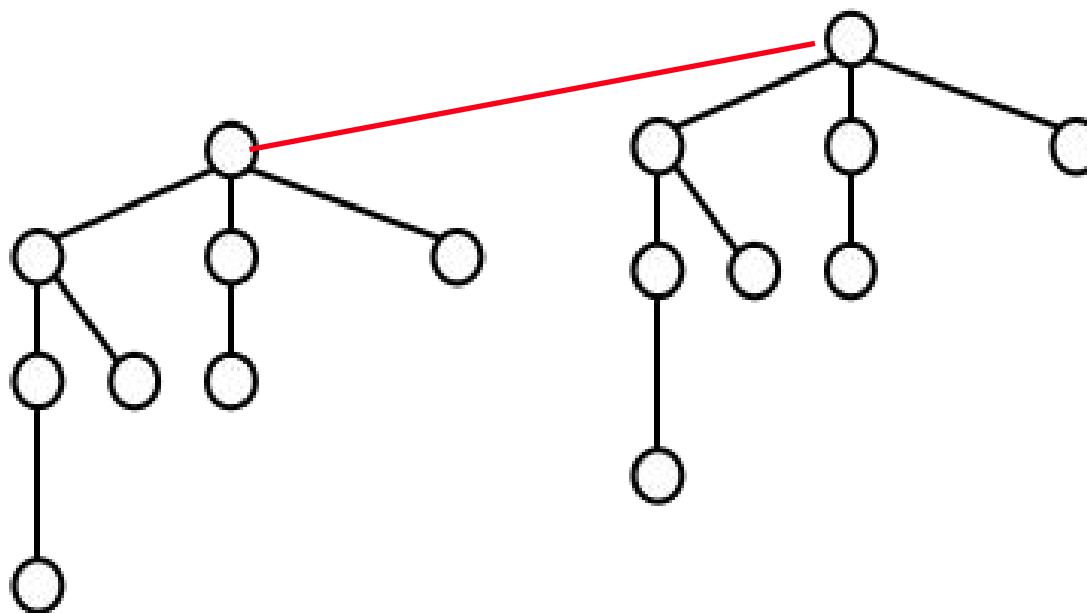
- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有二项树，按度数排序（从小到大）。
- 然后，两个度数都为 k 的二项树，可以合并成一个度数为 $k+1$ 的二项树。



二项堆上操作——UNION



- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有二项树，按度数排序（从小到大）。
- 然后，两个度数都为 k 的二项树，可以合并成一个度数为 $k+1$ 的二项树。



代价 $O(1)$

二项堆上操作——UNION

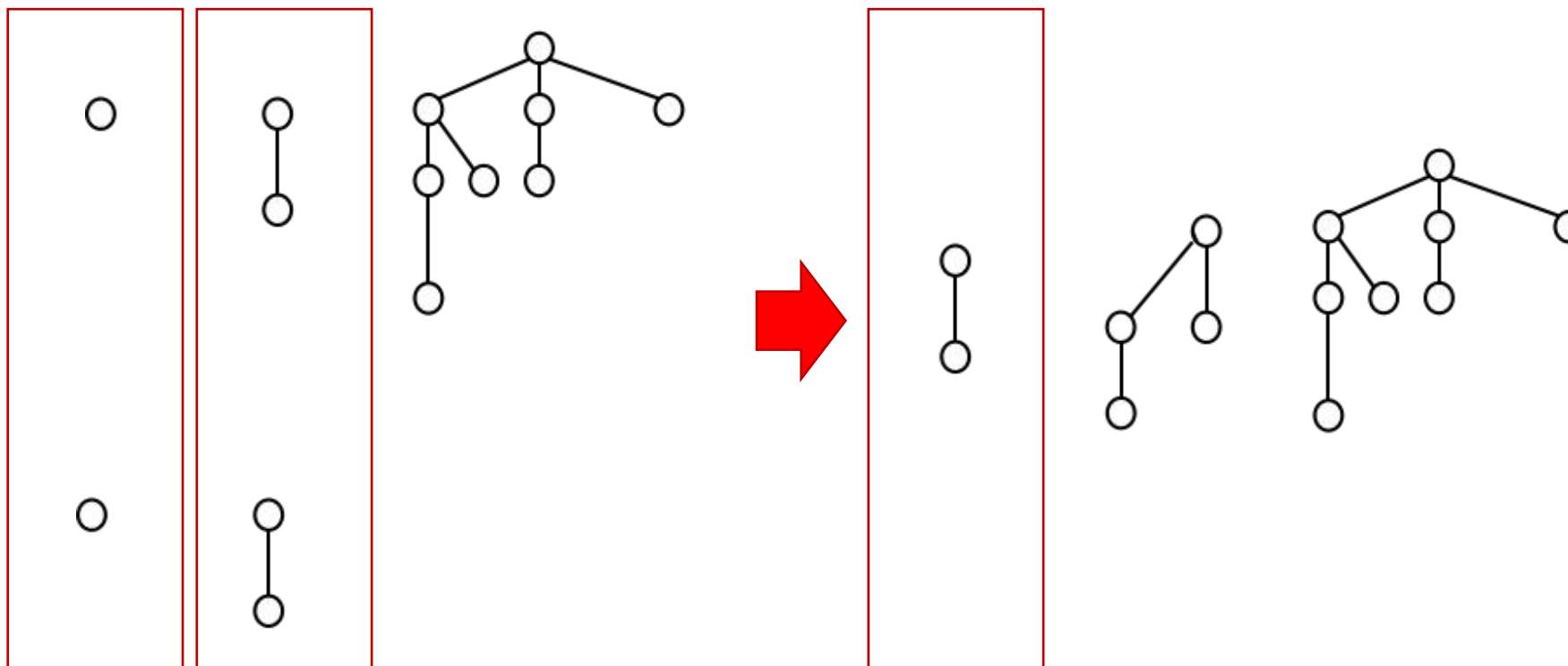


- 于是， H_1 和 H_2 中按度数从小到大合并二项树的时候，对于度数为 k 的情况有三种情况：
1. 度数为 k 的二项树 H_1 和 H_2 中都有1个，合并成1个度数为 $k+1$ 的二项树；
 2. 度数为 k 的二项树 H_1 或 H_2 中有1个，然后有2个度数为 $k-1$ 的二项树合并出来的1个度数为 k 的二项树，合并成1个度数为 $k+1$ 的二项树；
 3. 度数为 k 的二项树在 H_1 或 H_2 或度数为 $k-1$ 的二项树合并出来的度数为 k 的二项树里面只有1个，不操作；
 4. 度数为 k 的二项树没有，不操作。

二项堆合并示例



□ 以包含11个元素的堆和包含3个元素的堆合并为例



二项堆上操作——UNION

- 假设 H_1 和 H_2 中分别有 n_1 和 n_2 个元素，那么他们的二项树数量为 $O(\log n_1)$ 和 $O(\log n_2)$ 。于是， UNION代价 $O(\log n_1 + \log n_2)$

二项堆上其他操作

- ❑ **INSERT**: 等价于合并一个元素数量为1的堆, 复杂度 $O(\log n)$;
- ❑ **MINIMUM**: 在所有二项树的根中找最小, 有 $O(\log n)$ 个二项树, 所以复杂度 $O(\log n)$;
- ❑ **EXTRACT-MIN**: 在所有二项树的根中找最小进行删除, 会得到一个二项堆 H' , 合并原二项堆所剩下的二项树所组成的堆 H'' 和 H' , 复杂度 $O(\log n)$;
- ❑ **DECREASE-KEY**: 将减少值堆元素在它所在二项树上往上升, 复杂度 $O(\log n)$;
- ❑ **DELETE**: 将该元素的键值降低到比当前堆中最小值还小, 利用**DECREASE-KEY**, 然后**EXTRACT-MIN**, 复杂度 $O(\log n)$

二项堆改进

- ❑ 在二项堆基础上的重大改进，核心目标是降低某些关键操作的摊还时间复杂度，特别是那些在图算法（如 Dijkstra、Prim）中频繁调用的 **DECREASE-KEY** 和 **UNION**。
- ❑ 在斐波拉契堆里面，所有根用一个链表链起来，同时设定一个min指针指向最小值，所以**MINIMUM**代价是 $O(1)$
- ❑ 然后，相对于二项堆优化思路的核心在于：**懒惰 (Lazy) 操作**和**更宽松的树结构**

懒惰合并

□ 懒惰合并 (Lazy Merging)

- 二项堆: 合并时立即执行二进制加法, 逐阶链接同阶树, 代价 $O(\log n)$ 。
- 斐波那契堆: 合并两个堆时, 仅仅将它们的根链表拼接成一个链表, 不做任何树的合并, 因此 **UNIOIN** 操作只需 $O(1)$ 时间。

懒惰插入

- 懒惰插入

- 插入新元素时，直接将它作为一棵单节点树，添加到根链表中，而不尝试合并到现有树中。

- INSERT代价也是 $O(1)$ 。

更宽松的树结构

- 二项堆中每棵二项树是严格定义的：度数为 k 的树有固定形态，且根有 k 个子树（分别是 $B_0, B_1, \dots, B_{k-1}$ ）。
- 斐波那契堆中的树不要求根的子树按阶排序，只要**每个节点最多失去一个孩子**（通过标记机制维持）。

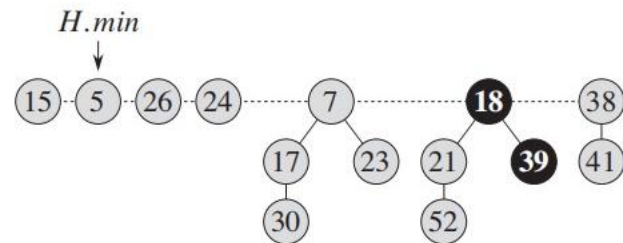
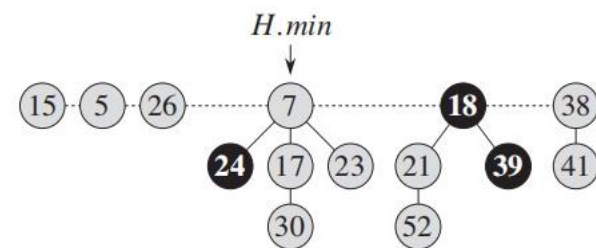
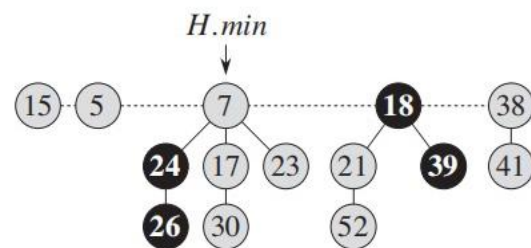
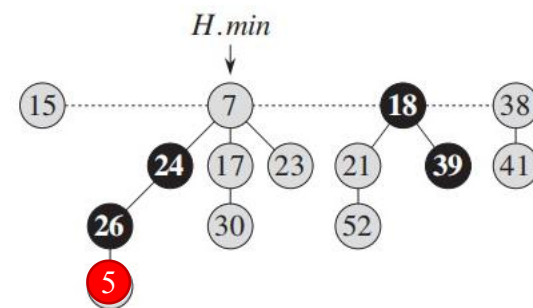
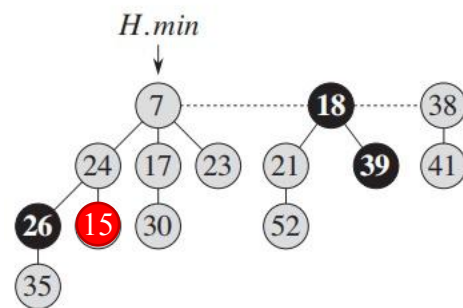
斐波拉契堆操作——DECREASE-KEY

级联切割

- ❑ **DECREASE-KEY**后，如果比父节点小，直接切断该节点与父节点的连接，并将该节点所在的子树整个移至根链表。
- ❑ 为了保持后续操作的摊还效率，对父节点做标记：如果父节点已经失去过一个孩子（标记为TRUE），则也将其切断并移至根链表，这个行为递归向上（级联切割）。
- ❑ 当一个元素被移至根链表，标记清为FALSE；一个元素初始状态标记都为FALSE

斐波拉契堆操作——DECREASE-KEY

示例



斐波拉契堆操作——DECREASE-KEY

代价分析

- 最坏情况下，单次操作的代价可能达到 $O(\log n)$ ；但是摊还代价为 $O(1)$

斐波拉契堆的DECREASE-KEY摊还分析

- 给定斐波拉契堆 H ，假设 $t(H)$ 是根链表中的树的数量， $m(H)$ 是被标记的节点数，那么定义势函数如下

$$\Phi(H) = t(H) + 2m(H)$$

- **DECREASE-KEY**每切断一个节点与其父节点连接，它为代价 $O(1)$ ，增加一棵树 ($t(H)$ 加 1) ，若切掉的节点原来有标记则 $m(H)$ 减 1 (标记被清除)
- 每次级联切割中，每切断一个节点与其父节点连接操作，要么减少一个标记节点 ($m(H)$ 减 1) ，要么将一个未标记节点变为标记 ($m(H)+1$)

斐波拉契堆的DECREASE-KEY摊还分析

- 假设斐波拉契堆 H 原本的势能为 $\Phi(H) = t(H) + 2m(H)$
- 一次**DECREASE-KEY**里做了 c 次切割，花了实际代价为 c ；
- 每次切割会增加一个树，所以**DECREASE-KEY**后的斐波拉契堆有 $t(H) + c$ 个树；第一次切割未必会减少标记，最后一次切割可能会增加一个标记，其他每次切割同时会消除一个标记，所以会 $m(H) - (c - 2)$

斐波拉契堆的DECREASE-KEY摊还分析

- 于是，势能变化为 $\Delta\Phi(H) = t(H) + c + 2(m(H) - c + 2) - (t(H) + 2m(H)) = 4 - c$
- 因此，一次DECREASE-KEY摊还代价为实际代价c加上势能变化4-c，也就是4，也就是O(1)

斐波拉契堆操作——EXTRACT-MIN

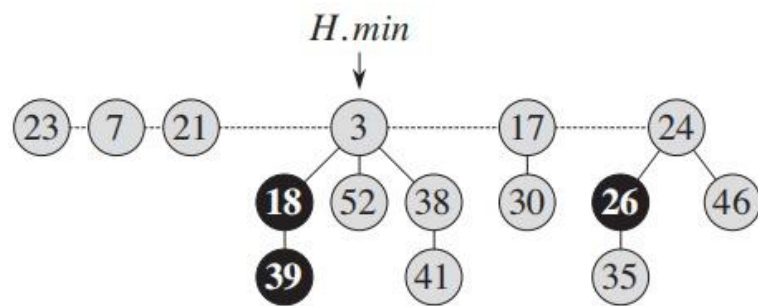
- 首先，利用根链表里面min指针，返回最小值并移除该节点；如果 min 有子节点，将所有子节点依次插入到根链表中（每个子节点成为一个新的根节点），并清空 min 的孩子列表；
- 由于之前的 insert、decrease-key 等操作都是“懒惰”的，根链表中可能有很多棵树（甚至 $O(n)$ 棵）。这一步的目标是将根链表中所有具有相同度数 (degree) 的树合并，直到每个度数最多只有一棵树。

斐波拉契堆操作——EXTRACT-MIN

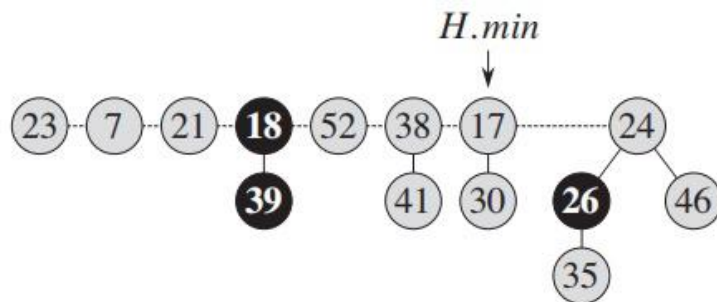
- 首先，利用根链表里面min指针，返回最小值并移除该节点；如果 min 有子节点，将所有子节点依次插入到根链表中（每个子节点成为一个新的根节点），并清空 min 的孩子列表；
- 由于之前的 insert、decrease-key 等操作都是“懒惰”的，根链表中可能有很多棵树（甚至 $O(n)$ 棵）。这一步的目标是将根链表中**所有具有相同度数的树合并**，直到每个度数最多只有一棵树。
- 树合并：使键值较小的成为新的根，另一个成为其孩子。

斐波拉契堆操作——EXTRACT-MIN

□ 初始状态:

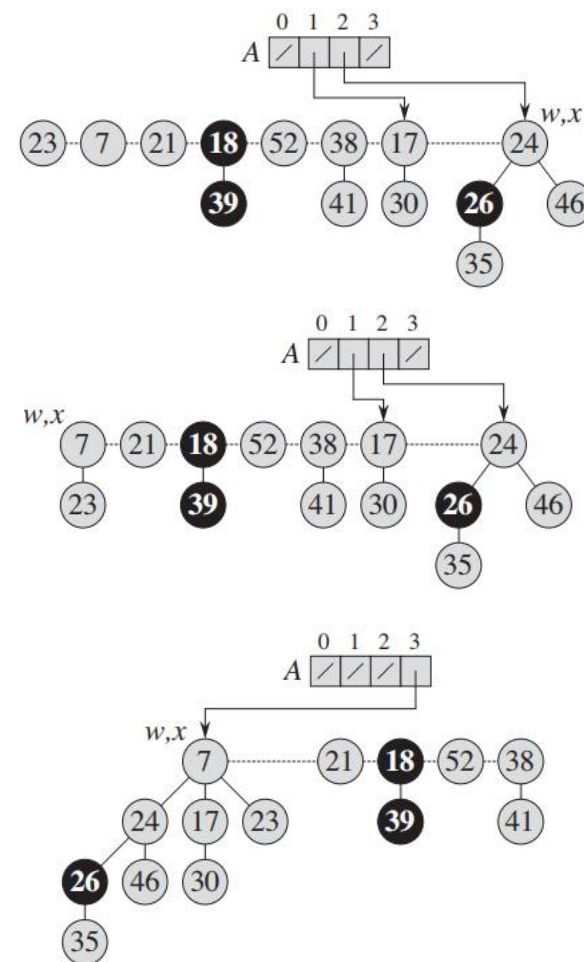
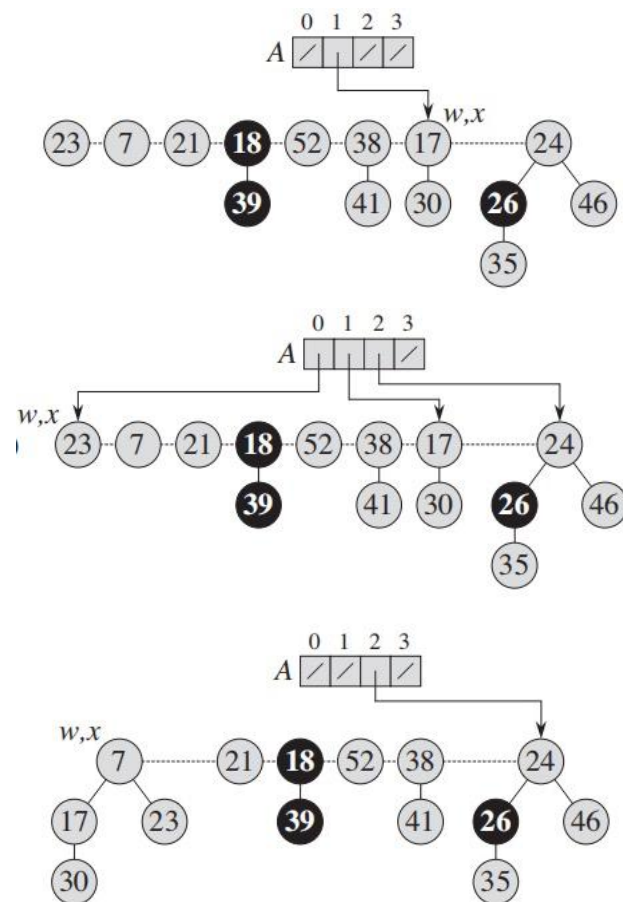


□ 返回并删去最小值:



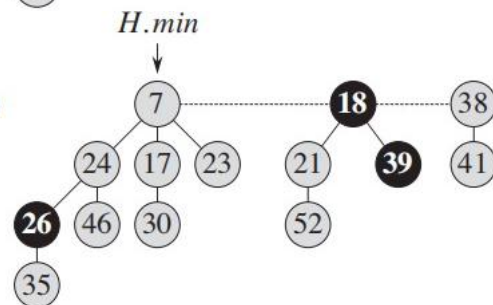
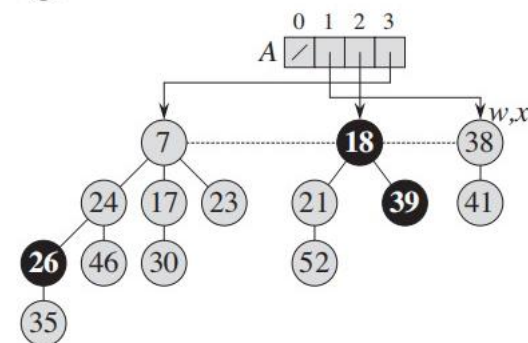
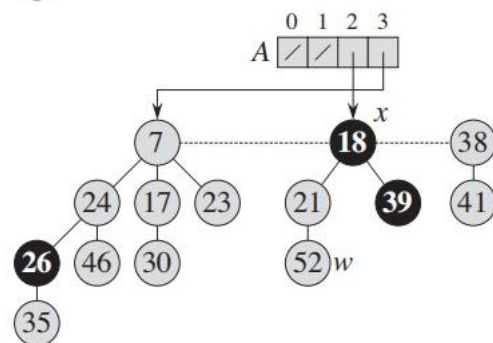
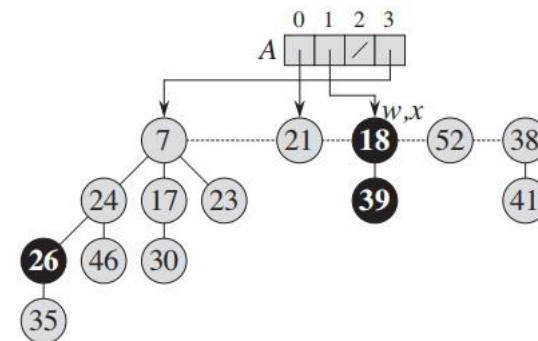
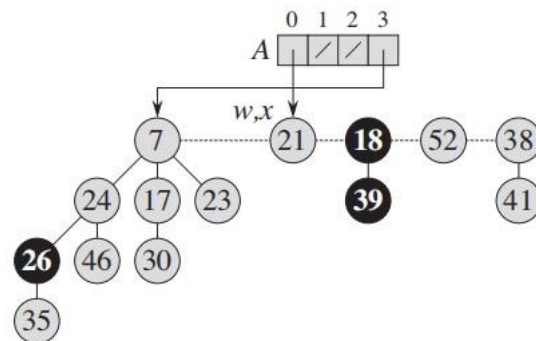
斐波拉契堆操作——EXTRACT-MIN

□ 树按照度数合并:



斐波拉契堆操作——EXTRACT-MIN

□ 树按照度数合并:



斐波拉契堆的EXTRACT-MIN摊还分析

- 可以证明，一个 n 个元素斐波拉契堆里面树度数上界 $D(n)=O(\log n)$ 。
- 那么**EXTRACT-MIN**过程前的 $t(H)$ 个合并到 $D(n)+1$ 个树上，实际代价会是 **$O(t(H) + D(n))$**
- **最坏情况：**若之前执行了大量 INSERT 和 DECREASE-KEY 操作而没有调用 EXTRACT-MIN，则根链表可能积累 $\Omega(n)$ 棵树（例如连续插入 n 个元素，每个成为独立一棵树）。此时合并步骤的代价为 $O(n)$ 。
- 因此 单次 EXTRACT-MIN 最坏时间复杂度为 **$O(n)$**

斐波拉契堆的EXTRACT-MIN摊还分析

- 给定斐波拉契堆H，假设 $t(H)$ 是根链表中的树的数量， $m(H)$ 是被标记的节点数，那么定义势函数如下

$$\Phi(H) = t(H) + 2m(H)$$

- 于是，**EXTRACT-MIN**过程后树的数量为 $D(n)+1$ ，同时**EXTRACT-MIN**过程不会增加标记点，所以**EXTRACT-MIN**过程后势能是 $D(n)+1+2m(H)$ ，所以势能变化为 $D(n)+1+2m(H)-t(H)-2m(H)=D(n)+1-t(H)$
- 所以**EXTRACT-MIN**摊还代价为 $O(t(H) + D(n))+D(n)+1-t(H)$ ，也就是 $O(D(n))=O(\log n)$

斐波拉契堆形状

- 在斐波那契堆中，一棵度数为 k 的树至少包含 F_{k+2} 个节点，其中 F 斐波那契数列 ($F_0=0$, $F_1=1$, $F_2=1$, $F_3=2$, ...)



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——